



Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia Informática

Unidade Curricular de Requisitos e Arquiteturas de Software

2025/2026

Fase 2

Diogo Miguel Pinto pg61515
Gabriel Veloso Antunes pg60259
Guilherme Pinto Pinho pg60263
Mariana Miguel Pinto pg61534
Miguel Freixo Machado pg60284
Janeiro, 2026

RAS

Data da Receção	
Responsável	
Avaliação	
Observações	

Fase 2

Diogo Miguel Pinto pg61515

Gabriel Veloso Antunes pg60259

Guilherme Pinto Pinho pg60263

Mariana Miguel Pinto pg61534

Miguel Freixo Machado pg60284

Janeiro, 2026

Índice

1. Introdução	1
2. Manutenção Corretiva da Solução Atual	2
2.1. T-01 – Cancelar processamento sem recarregar a página	2
2.2. T-02 – Uniformização das mensagens de erro	2
2.3. T-03 – Tempos de aplicação dos filtros	3
2.3.1. Otimização aplicada (short-circuit no Apply)	3
2.4. T-04 – Correspondência entre pré-visualização e resultado final	4
2.5. T-05 — Encriptação, Partilha com Permissões e Revogação de Acessos	4
2.5.1. Encriptação em trânsito e em repouso	4
2.5.2. Modelo de partilha: estrutura dos links e propagação do shareId	5
2.5.3. Propagação do share no editor e controlo de permissões	5
2.5.4. Revogação e comportamento em tempo real	8
2.6. T-06 – Reordenar ferramentas e reaplicar sequência	9
2.7. T-07 – Limite de operações diárias (ferramentas avançadas)	10
2.8. T-08 – Remoção de conta e dados (RGPD)	10
2.9. T-09 – Escolha do formato de download	10
2.10. T-10 – Cancelar processamento com destaque visual	11
2.11. T-11 — Backups Automáticos Diários	11
2.12. Gestão de Edições: Undo, Reset e Previews	12
2.12.1. Solução implementada	12
2.12.1.1. Undo	12
2.12.1.2. Reset	12
2.12.2. Nota sobre o modo results	13
3. Novo Caso de Uso: Partilha e Colaboração	13
3.1. RF50 – Forçar Guardar	13
3.1.1. Metadados	13
3.1.1.1. Estado global “alterações não guardadas”	13
3.1.1.2. Sincronização robusta do estado “dirty” e versão do projeto (RNF46)	14
3.1.1.3. Marcação de alterações como “não guardadas” (alterações locais)	14
3.1.1.4. Limpeza do estado (após Save/Default/Reset)	14
3.1.2. Bloqueio da partilha e feedback ao utilizador ->	15
3.1.2.1. Indicador visual no botão “Share”	15
3.1.2.2. Bloqueio efetivo da criação de links	15
3.1.3. Verificação do critério de aceitação	15
3.2. RF41 — Edição colaborativa em tempo real (Should)	16
3.2.1. Mecanismo	16
3.2.2. Efeito prático	16
3.3. RNF65 – Limite de utilizadores num Projeto (Should)	17
3.3.1. Modelo de presença e expiração automática	17
3.3.2. Validação do limite com base no tipo do dono do projeto	17
3.3.3. Middleware de enforcement e códigos de erro	17
3.3.4. Integração nas rotas relevantes (Projects-MS)	17
3.3.5. API Gateway: propagação correta de identidade e compatibilidade com modo partilhado	18
3.3.6. Frontend: heartbeat e mensagem ao utilizador (fit criterion)	18
3.3.7. Resultado final	18
3.4. RF53 – Acesso por link com permissões (Must).	18
4. Novo Caso de Uso: Presets	19
4.1. Requisitos e Objetivo	19
4.2. Modelo de dados e persistência	19
4.3. API e autenticação (RF70)	19

4.4.	Integração no frontend e desempenho (RNF60/RNF66)	19
4.5.	Edição, remoção e partilha (RF67/RF68/RF39)	20
5.	Novas Funcionalidades com IA	20
5.1.	Novo Caso de Uso: Assistente IA na edição (UC06 / RF44)	20
5.1.1.	Backend (Projects-MS): geração de sugestões	20
5.1.1.1.	Sugestões adaptativas ao conteúdo da imagem (features + imgId)	20
5.1.1.2.	RF61 — Preview visual das sugestões (pré-visualização por “receita”)	21
5.1.2.	API Gateway: encaminhamento seguro para o Projects-MS	22
5.1.3.	Frontend: UI do assistente e edição de sugestões	22
5.1.4.	Integração na página do projeto	23
5.1.5.	Validação (fit criteria)	23
5.2.	RF40 – Generative Expand (expand_ai)	23
5.2.1.	Parâmetros suportados e comportamento	23
5.2.2.	Integração no backend (Tools + RabbitMQ)	24
5.2.3.	Serviço generative-mock para testes locais	24
5.2.4.	Alterações de infraestrutura (docker-compose)	24
5.2.5.	Integração no frontend (Toolbar + Preview/Apply)	24
5.2.6.	Validação (fit criteria)	25
6.	Melhorias implementadas ao projeto	25
6.1.	Melhorias no processamento de projetos	25
6.2.	Reestruturação da contagem de ferramentas avançadas	25
6.3.	Novo fluxo de processamento	26
6.3.1.	Utilizador adiciona ferramentas (IA e não-IA)	26
6.3.2.	Ao clicar em “Apply”, o sistema verifica	26
6.3.3.	A pipeline completa é aplicada	26
6.3.4.	No caso de cancelamento	26
6.4.	Cancelamentos e reembolsos de operações	26
6.5.	Benefícios da nova abordagem	26
6.6.	As ferramentas não-IA seguem esta lógica?	27
6.6.1.	No que diz respeito a custos	27
6.6.2.	No que diz respeito ao comportamento da pipeline	27
6.7.	Modo escuro (RNF59)	27
7.	Design Decisions	28
7.1.	Padrão Arquitetural	28
7.2.	Tabela de alocação funcional dos requisitos	29
7.3.	Tabela de alocação dos requisitos não funcionais	31
8.	Building Block View	32
9.	Runtime View - Colaboração (UC07)	34
10.	Runtime View - Cancelamento	36
11.	Deployment View	38
12.	Análise Crítica	41
13.	Conclusão	45

Lista de Figuras

Figura 1	Diagrama de Componentes referente ao <i>Building Block View</i>	32
Figura 2	Diagrama de Sequência - Colaboração	34
Figura 3	Diagrama de Sequência - Cancelamento	36
Figura 4	Diagrama de Instalação - Deployment View	38

Lista de Tabelas

Tabela 1	29
Tabela 2 Tabela de alocação Funcional de requisitos	30
Tabela 3 Tabela de alocação dos requisitos não funcionais	31
Tabela 4	41
Tabela 5 Tabela das tarefas implementadas relativas ao UC07	42
Tabela 6	43
Tabela 7 Tabela dos requisitos implementados relativos ao UC07	44

1. Introdução

Este documento foi elaborado de modo a registar o processo de expansão de funcionalidades do sistema PictuRAS, uma plataforma de processamento e edição de imagens.

Neste relatório são descritas as fases de manutenção corretiva da solução original que nos foi disponibilizada, de modo a corrigir inconsistências presentes ou defeitos não identificados no documentos de requisitos da solução original, fase de implementação de novos casos de uso com foco particular no UC07 - Partilhar Projeto e por fim, esta que assenta sobre a já referida expansão das funcionalidades já implementadas e na fase de exposição da solução arquitetural, fase esta que auxilia na explicitação da estrutura do sistema demonstrando de que modo é que as estratégias mencionadas nas outras fases são atendidas de um ponto de vista mais técnico e observacional.

No final do relatório é apresentado de forma mais intuitiva o estado de implementação dos diferentes requisitos, quer sejam Funcionais ou Não Funcionais, de forma a conseguir consultar mais facilmente o grau de completude da nova solução proposta.

2. Manutenção Corretiva da Solução Atual

A manutenção corretiva da solução atual incidiu prioritariamente sobre as tarefas de elevada prioridade identificadas na Tabela 62 do documento de requisitos. Para além destas, foram também endereçados problemas adicionais detetados no comportamento da aplicação.

2.1. T-01 – Cancelar processamento sem recarregar a página

A tarefa **T-01** consistiu em implementar a capacidade de cancelar um processamento em curso sem necessidade de recarregar a página, garantindo ainda que o estado visual e funcional do sistema fosse restaurado em menos de um segundo. Esta funcionalidade exigiu intervenções tanto no **backend** como no **frontend**, bem como no mecanismo de comunicação em tempo real do sistema.

No backend, foi criado um endpoint DELETE `/projects/:uid/:pid/process` que permite cancelar um processamento em curso sem destruir resultados anteriores. Cada execução possui um identificador sequencial (token da execução), controlado pelos campos **activeToken**, **committedToken** e **prevCommittedToken** no projeto.

Ao iniciar um processamento:

- **prevCommittedToken** recebe o valor atual de **committedToken**;
- **activeToken** é incrementado e usado como token da nova execução.

Ao cancelar:

- é capturado o token da execução a cancelar (**cancelTok** = **activeToken**);
- **activeToken** é incrementado, invalidando outputs tardios;
- se **committedToken** == **cancelTok**, é feito rollback para **prevCommittedToken**;
- são removidos apenas os resultados associados ao token cancelado;
- são libertadas operações avançadas pendentes dessa execução.

Desta forma, se o utilizador aplicar A, iniciar B e cancelar B, o sistema volta automaticamente a apresentar os resultados de A, mantendo coerência total.

No frontend, o cancelamento usa a referência `ignoreUpdatesRef` para ignorar eventos WebSocket tardios. Contudo, a robustez principal é garantida no backend, onde cada output é validado contra o **activeToken** atual antes de ser persistido.

2.2. T-02 – Uniformização das mensagens de erro

A tarefa T-02 visou corrigir um problema comum em aplicações distribuídas: mensagens de erro inconsistentes, pouco informativas ou com textos técnicos que não ajudam o utilizador final. O objetivo central foi criar um sistema unificado e inteligível para todos os erros, independentemente da sua origem — serviços internos, IA, rede ou validações do API Gateway.

Para isso, foi criado um módulo centralizado denominado `lib/error-messages.ts`, responsável por transformar qualquer erro (incluindo erros Axios, erros emitidos pelo backend, códigos das ferramentas de IA e falhas de rede) num par estruturado `{ title, description }`. Este módulo introduz a função `getErrorMessage(context, error)`, onde o contexto (`"auth-login"`, `"project-process"`, `"billing"`, etc.) permite adaptar a explicação ao fluxo atual do utilizador, garantindo mensagens coerentes e em português claro.

Além disso, erros específicos das ferramentas de IA passaram a ser tratados por uma função dedicada, `getAiErrorMessage`, que mapeia códigos numéricos provenientes do processamento (como 1100, 2001 ou 1800) para explicações compreensíveis e adequadas à ferramenta utilizada. Isto contribuiu para uma experiência mais intuitiva e alinhada com o comportamento real do sistema.

Após a criação do módulo, foram substituídas as ocorrências mais relevantes de `error.message` (toasts e componentes) por chamadas a `getErrorMessage/getAiErrorMessage`, cobrindo autenticação, faturação, gestão de projetos, upload/download, processamento, reordenação, cancelamento e pré-visualização, incluindo falhas de rede (`axios.isAxiosError(error) && !error.response`).

Como resultado, o sistema passou a apresentar mensagens de erro uniformes, claras e com sugestões práticas de resolução, eliminando mensagens técnicas expostas ao utilizador e garantindo que todas as interações refletem um comportamento previsível e profissional.

2.3. T-03 – Tempos de aplicação dos filtros

A tarefa T-03 teve como objetivo monitorizar, validar e tornar visíveis os tempos de aplicação dos filtros do sistema, assegurando que estes respeitam os limites definidos nos requisitos. Para isso, foi introduzida uma medição rigorosa do tempo total de processamento e do tempo médio por passo, contabilizando o número de operações individuais realizadas durante a execução do pipeline.

Na prática, sempre que um utilizador inicia um processamento, o frontend regista o instante inicial com `performance.now()` e a cada atualização recebida via WebSocket — emitida pelo serviço de processamento — é incrementado o contador interno de passos. A combinação destes dois valores permite calcular métricas como:

- duração total do processamento;
- tempo médio por passo;
- comparação automática com limiares definidos no sistema.

Foram definidos dois limiares de referência: aproximadamente 5 segundos para filtros não baseados em IA e cerca de 60 segundos para ferramentas de IA. Quando estes limites são ultrapassados, o sistema gera automaticamente uma notificação (toast) informando o utilizador de que a operação demorou mais do que o esperado, sem contudo interromper o processamento.

2.3.1. Otimização aplicada (short-circuit no Apply)

Para cumprir explicitamente a componente “otimizar quando necessário”, foi introduzida uma otimização simples no backend: quando o utilizador clica em Apply mas o pipeline de ferramentas não sofreu qualquer alteração desde o último processamento confirmado, o sistema evita recomputação completa.

No Projects-MS, o endpoint `POST /:user/:project/process` calcula um fingerprint determinístico do pipeline (ordem das tools + procedure + params) através de serialização estável e hash SHA-256. Este fingerprint é comparado com o fingerprint do último estado comprometido (`committedTools`). Se forem iguais e existir um commit anterior (`committedToken > 0`), o processamento é “curto-circuitado”:

- não é criada uma nova execução (não há incremento de `activeToken` para processar);
- não é publicado nenhum pedido no RabbitMQ para os workers;
- o backend responde imediatamente como concluído, evitando publicar jobs e mantendo os resultados atuais.

Este mecanismo reduz o tempo de Apply para o tempo de rede e evita carga desnecessária (fila + workers + storage), sendo particularmente relevante quando o utilizador tenta aplicar repetidamente o mesmo conjunto de ferramentas (por exemplo após alternar entre modos de visualização).

2.4. T-04 – Correspondência entre pré-visualização e resultado final

Inicialmente, existiam discrepâncias entre a pré-visualização apresentada durante a fase de edição e o resultado final produzido após o processamento do projeto. Estas diferenças deviam-se a inconsistências na forma como os ficheiros eram geridos internamente, nomeadamente na utilização de imagens temporárias (preview) e imagens finais (results), bem como na gestão de caminhos e versões no storage.

Para resolver este problema, o sistema passou a tratar explicitamente duas fases distintas:

- 1. Pré-visualização: utiliza imagens temporárias próprias, correspondentes apenas à aplicação de uma ferramenta isolada;
- 2. Processamento final: segue rigorosamente a sequência completa de ferramentas definidas pelo utilizador, sempre com as imagens reais e persistentes do projeto.

Foram corrigidos os pontos onde a pipeline ainda recorria a ficheiros “og_img_uri” em vez das versões atualizadas (“new_img_uri”) e ajustada a lógica de substituição dos ficheiros no storage. Agora, cada passo do processamento final utiliza exatamente o resultado do passo anterior, garantindo correspondência total entre o comportamento observado na pré-visualização e o resultado final.

Sempre que um novo processamento é iniciado, os resultados anteriores não são removidos de imediato. Estes permanecem associados ao committedToken anterior e continuam a ser servidos ao frontend.

Dito isto, apenas quando uma nova execução termina com sucesso é que o committedToken é atualizado, passando o modo results a apontar para o novo conjunto de imagens e, em caso de cancelamento, apenas os resultados associados ao token cancelado são removidos, sendo restaurado automaticamente o conjunto de resultados anterior.

2.5. T-05 — Encriptação, Partilha com Permissões e Revogação de Acessos

A tarefa T-05 abordou dois aspetos: (i) proteção de dados (em trânsito e em repouso) & (ii) partilha de projetos por link com permissões explícitas (read/edit) e revogação efetiva, refletida no frontend sem necessidade de refresh manual.

2.5.1. Encriptação em trânsito e em repouso

A comunicação entre browser, API Gateway e microserviços é feita sobre HTTPS/TLS através de um reverse proxy Nginx (certificado self-signed no ambiente de desenvolvimento). O mesmo canal protege as ligações WebSocket, garantindo confidencialidade e integridade dos dados em trânsito.

Em repouso, foram aplicadas duas camadas:

- Base de dados: o email do utilizador deixou de ser armazenado em texto claro, passando a ser cifrado com AES-256-GCM. Em paralelo, é guardado um hash SHA-256 irreversível para suportar unicidade/pesquisa sem expor o valor real.
- Storage (MinIO): os ficheiros são protegidos com encriptação server-side (SSE-KMS), mantendo-se o acesso via URLs pré-assinadas.

2.5.2. Modelo de partilha: estrutura dos links e propagação do shareId

Cada projeto pode agora ter múltiplos links de partilha, guardados diretamente no documento do projeto, num campo `sharedLinks`. Cada entrada tem a forma:

- `id`: identificador único (UUID) usado como `shareId` no URL;
- `permission`: “read” ou “edit”;
- `createdAt`: data de criação do link;
- `revoked`: flag booleana que indica se o link foi revogado.

O serviço `Projects-MS` expõe endpoints específicos para gerir e resolver estes links:

- `POST /:user/:project/share` – cria um novo link (read ou edit) e devolve o URL público `FRONTEND_BASE_URL/share/:shareId`;
- `GET /share/:shareId` – resolve um link para { `projectId`, `ownerId`, `permission`, `projectName` }, devolvendo 410 se o link estiver revogado;
- `GET /share/:shareId/project` – devolve o projeto completo (ferramentas + imagens mais recentes), com verificação do estado de revogação, exigindo autenticação (JWT).

No API Gateway, o endpoint de resolução do link `/projects/share/:shareId` é público, permitindo que qualquer utilizador valide o link e conheça apenas metadados mínimos. Já o endpoint `/projects/share/:shareId/project` exige autenticação, devolvendo 401 quando o utilizador não está autenticado. Isto garante que um utilizador anónimo consegue abrir a página “Shared project”, mas não consegue obter conteúdo do projeto (nome real, imagens e ferramentas) sem login/register.

No frontend, a página de partilha corresponde à rota `/share/[linkId]` => que funciona em duas fases:

- (1) resolve público do link via `GET /projects/share/:shareId`, para obter `permission/ownerId/projectId` e validar revogação;
- (2) apenas após autenticação, obtém o projeto completo via `GET /projects/share/:shareId/project`.

O componente mantém polling a cada 5 s sobre o resolve do link. Quando a API começa a devolver 410 (link revogado), o estado local muda para error e a UI passa a mostrar a mensagem “Invalid or revoked link” sem necessidade de refresh manual.

Se o link tiver permissão “edit”, esta mesma página gera o URL do editor completo:

- `/dashboard/:projectId?owner=:ownerId&share=:shareId` => e usa esse URL tanto para abrir diretamente o editor (se o utilizador já estiver autenticado) como para construir os links de Login / Register com `?next=`.

2.5.3. Propagação do share no editor e controlo de permissões

Quando o convidado abre o editor através de um link com permissão “edit”, a página do projeto (`/dashboard/[pid]`) lê os parâmetros da query:

- `owner` – identifica o verdadeiro dono do projeto;
- `share` – identifica o link de partilha em uso.

Estes valores são propagados para todas as chamadas ao backend através das queries e mutations de projetos:

- `useGetProject / fetchProject` $\Rightarrow$ `GET /projects/:pathUid/:pid + query (? share=...&owner=...)` onde `pathUid = ownerId` quando existe `shareId`, caso contrário `pathUid = uid` (dono).
- `useGetProjectResults` $\Rightarrow$ `GET /projects/:pathUid/:pid/process/url` com o mesmo par `owner/share`.
- `previewProjectImage`, `addProjectTool`, `processProject` e restantes operações de edição $\Rightarrow$ todas constroem a query string com `owner` e `share` via `buildQuery({ ownerId, shareId })`.

No API Gateway, estes parâmetros são reenviados para o serviço Projects-MS:

- `GET /projects/:ownerId/:projectId?share=:shareId`
- `POST /projects/:ownerId/:projectId/tool?share=:shareId`
- `POST /projects/:ownerId/:projectId/preview/:imgId?share=:shareId`
- `POST /projects/:ownerId/:projectId/process?share=:shareId`
- ...

Um detalhe adicional necessário para robustez do modo partilhado é garantir que os parâmetros de contexto (`owner/share`) não se perdem em navegação client-side. Componentes que alteram apenas o estado da UI via query string (ex.: alternar `mode=edit/results` ou `view=grid/carousel`) devem preservar o `shareId`; caso contrário, o frontend deixa de construir a query e o `pathUid` corretamente, originando pedidos inválidos ao Projects-MS e erros de carregamento do projeto.

No frontend, isto foi garantido propagando explicitamente `share` (e `owner`) na construção do URL de navegação do editor (ex.: `ModeToggle`).

checkSharePermission

- se não existir `?share`, considera que o acesso é feito pelo dono;
- se existir, procura o projeto por `shareId`, valida se o link existe, não está revogado e se `:user` e `:project` do path coincidem com o dono e com o projeto;
- injeta em `req.sharedPermission` o valor “read” ou “edit”.

requireEditPermission

- está ligado apenas às rotas de escrita (adicionar imagens, ferramentas, reordenar, fazer preview, processar, etc.);
- se não houver `req.sharedPermission` (caso do dono, sem `share`), o pedido passa;
- se houver `req.sharedPermission === “edit”`, o pedido passa;
- caso contrário, responde com 403 “Share link has no edit permission”.

Desta forma:

- links “read” permitem apenas operações de leitura (carregar projeto, imagens e resultados);
- links “edit” permitem também todas as operações de edição, processamentos e pré-visualizações;
- o dono continua a ter acesso total sem depender de qualquer link de partilha.

No frontend, para alinhar a experiência com o modelo de permissões, o editor distingue explicitamente `canEdit`. Quando um projeto é aberto via link read, a página continua a renderizar o editor em modo `'edit'` para manter consistência de UI, mas todas as ações de escrita ficam bloqueadas: seleção/edição de ferramentas, `Save/Preview`, `Apply`, e sincronização do estado `'dirty'`. Desta forma, em modo read não existem pedidos de escrita que resultem em 403, e o utilizador tem uma experiência claramente `'read-only'`.

Um detalhe importante no modo partilhado é a separação entre o utilizador autenticado (JWT) e o identificador presente no path. Quando existe `?share=:shareId`, o parâmetro `:user` nas rotas internas representa o `ownerId` (dono real do projeto), enquanto o JWT corresponde ao utilizador convidado autenticado. Por esse motivo, o middleware de autenticação do API Gateway valida a assinatura/expiração do token, mas não impõe a regra `JWT.id == :user` quando existe `shareId` presente (essa validação continua a existir no modo “normal”, sem share).

A verificação de acesso ao projeto (incluindo revogação e permissões read/edit) fica então centralizada no Projects-MS, através do `shareId` e dos middlewares de autorização. Assim, o API Gateway garante autenticação (token válido) e o Projects-MS garante autorização (pode ou não aceder/editar aquele projeto via link).

Além disso, foi necessário introduzir a distinção entre `owner` (dono do projeto) e `runner` (utilizador que iniciou a ação). No modo partilhado com permissão “edit”, o convidado pode lançar previews e processamentos sobre um projeto que pertence ao `owner`. Contudo, as atualizações em tempo real (WebSocket) devem ser enviadas para quem iniciou a operação — caso contrário, o frontend do convidado pode permanecer em estado “processing” por não receber os eventos.

Para resolver isto, o API Gateway passa a encaminhar explicitamente a identidade do utilizador autenticado para o Projects-MS através do header `X-Caller-Id`. No Projects-MS, esse identificador é usado como `runnerId` (ou, na ausência do header, é obtido por decode do JWT) e é registado nos registos de processamento. Quando o pipeline reporta progresso/conclusão, o Projects-MS publica eventos WebSocket com o campo `user` igual ao `runnerId`, garantindo que a gateway de WebSockets emite o evento para a “room” correta do utilizador que iniciou a operação (incluindo convidados com link edit).

Por fim, para evitar perda silenciosa de alterações quando duas sessões tentam editar o mesmo projeto (ex.: `owner` e convidado em simultâneo), foi adotado um controlo de concorrência otimista baseado em versão. O Projects-MS incrementa um campo `version` do projeto a cada alteração bem-sucedida. O frontend envia a versão esperada no header `X-Project-Version` em operações de escrita (ex.: adicionar/remover ferramentas, reordenar, upload).

Se a versão no servidor já tiver avançado, o backend responde com 409 Conflict e devolve a versão atual no corpo da resposta (campo `serverVersion`). O frontend pode então atualizar a versão local e repetir a operação, evitando “lost updates” e corrupção de dados em edições simultâneas.

Em operações bem-sucedidas, o Projects-MS devolve a nova versão no header `X-Project-Version`, que é usada pelo frontend para atualizar o estado local.

Para garantir a robustez do protocolo de edição, pedidos de escrita que não incluam o header `X-Project-Version` são rejeitados com 428 Precondition Required, impedindo alterações sem controlo de concorrência.

No frontend, para evitar conflitos em operações consecutivas dentro do mesmo diálogo (por exemplo, revogar dois links seguidos), o componente de partilha mantém e utiliza sempre a versão mais recente do projeto. Para isso, a versão é obtida a partir da cache do React Query e complementada por um estado local (`vLocal`) que é atualizado imediatamente após cada resposta bem-sucedida via header `X-Project-Version`. Assim, a ação seguinte reutiliza automaticamente a versão recém-gerada, reduzindo conflitos 409 causados por estado desatualizado entre cliques sucessivos.

2.5.4. Revogação e comportamento em tempo real

Do lado do proprietário, a UI mostra a lista de links ativos (listProjectShareLinks) e permite revogar qualquer um através de DELETE /projects/:userId/:projectId/share/:shareId

Este pedido é encaminhado para o serviço Projects-MS, que:

- valida a versão esperada (X-Project-Version) para evitar lost updates;
- identifica o projeto pelos parâmetros :userId/:projectId e revoga o link :shareId (marca revoked = true);
- persiste a alteração na base de dados.

A partir desse momento:

- GET /projects/share/:shareId e GET /projects/share/:shareId/project passam a devolver 410 “Share link revoked”;
- qualquer pedido ao Projects-MS com ?share=:shareId é interceptado por checkSharePermission, que também devolve 410 quando o link está revogado;
- as rotas de escrita protegidas por requireEditPermission passam a recusar operações de edição associadas a esse link.

Na prática, isto traduz-se no seguinte comportamento observado no frontend:

Links de leitura (read)

A página /share/[linkId] continua a fazer polling a cada 5 s sobre o resolve do link. Mal o proprietário revoga o link, a chamada seguinte deteta o 410 e a UI muda automaticamente para o estado “Invalid or revoked link”, sem ser necessário recarregar o browser.

Links com edição (edit) já abertos no editor

Quando o convidado abre o editor através de um URL do tipo /dashboard/:pid?owner=:ownerId&share=:shareId, a página do projeto:

- lê os parâmetros owner e share e propaga-os para todas as queries/mutations (getProject, processProject, preview, etc.);
- enquanto existir shareId, ativa um “heartbeat” de partilha: a cada 10 s chama o resolve do link (GET /projects/share/:shareId) em background;
- se esta chamada falhar (por exemplo, devolvendo 410), o estado local shareRevoked passa a true.

Quando shareRevoked é verdadeiro, o editor mostra imediatamente, no topo da página, uma barra de aviso em vermelho: “O proprietário revogou o link. Já não podes editar este projeto.”

Além disso, se a página for recarregada ou se a query de carregamento do projeto (useGetProject) falhar com shareId presente, o ecrã principal apresenta um alerta específico: “Link de partilha revogado — Este link de partilha já não é válido. O proprietário revogou o acesso a este projeto.”

Indicador de projeto compartilhado na sidebar

Na área “Your projects”, foi ainda adicionada uma integração com o estado da partilha:

- quando o utilizador não tem projetos próprios e está a trabalhar num projeto aberto por link de partilha válido, a sidebar mostra uma entrada do tipo NomeDoProjeto (Shared, permission: read/edit) em vez de “Empty for now.”;
- esta entrada é construída a partir do hook `useResolveShareLink(shareId)`, que resolve o `shareId` para `{ projectId, ownerId, permission, projectName }` sem depender do carregamento do projeto completo;
- assim que o link é revogado e o backend passa a devolver 410, o resolve entra em estado de erro e a entrada (Shared) desaparece automaticamente, voltando a ser mostrada a mensagem “Empty for now.”.

Com estes mecanismos combinados — **revogação no backend, heartbeat no editor, mensagens específicas no carregamento do projeto e indicação clara na sidebar** — a revogação de partilhas passa a ser refletida em tempo quase real no frontend, de forma clara e previsível para o utilizador convidado.

2.6. T-06 – Reordenar ferramentas e reaplicar sequência

Esta tarefa consistiu em permitir ao utilizador reorganizar a ordem das ferramentas aplicadas a um projeto e fazer com que essa nova ordem fosse imediatamente refletida tanto no frontend como no backend. Isto é essencial dado que a sequência das ferramentas influencia diretamente o resultado final do processamento.

No frontend foi implementado uma interface de arrastar-e-largar (drag & drop) utilizando a biblioteca `@dnd-kit`, criando uma lista ordenável na secção “Applied Tools”. Cada ferramenta aplicada passa a ser representada como um item ordenável, e sempre que o utilizador altera a sua posição, o frontend:

- atualiza a ordem localmente de imediato, para garantir resposta visual instantânea;
- recalcula o campo `position` de cada ferramenta;
- envia a nova ordem para o backend através do endpoint `POST /projects/:user/:pid/reorder`.

No backend, essa nova ordem é recebida e persistida na base de dados, garantindo consistência entre UI e estado real do projeto. Além disso, a pipeline de processamento foi ajustada para garantir que os filtros são aplicados exatamente pela ordem que o utilizador definiu, eliminando inconsistências anteriores. Assim, sempre que o utilizador processa novamente o projeto, o sistema aplica a sequência de ferramentas na ordem exata definida após a reordenação.

Nesta arquitetura, a reordenação atualiza imediatamente a lista de ferramentas e a sua ordem persistida no backend (efeito instantâneo no editor), mas não dispara automaticamente um novo processamento final. A decisão é intencional: o modo `results` representa o último processamento completo concluído e só é atualizado quando o utilizador executa `Apply`. Assim, o requisito é satisfeito ao nível da atualização imediata do estado e pipeline (sequência efetiva de execução), enquanto a geração de novos resultados permanece uma ação explícita do utilizador para evitar custo computacional e consumo indevido de operações (especialmente em ferramentas de IA)."

2.7. T-07 – Limite de operações diárias (ferramentas avançadas)

A gestão do limite diário de operações avançadas (ferramentas IA) foi implementada de forma completa e integrada entre frontend, API Gateway e o serviço Users. O objetivo desta tarefa não era apenas bloquear o processamento quando o limite fosse atingido, mas garantir que:

- o limite é validado antes de cada processamento;
- o contador é atualizado após cada operação concluída;
- cancelamentos devolvem operações ao utilizador (quando aplicável);
- o frontend apresenta o número de operações restantes de forma correta.

Quando o utilizador ativa o processamento de um projeto que inclui ferramentas avançadas, o serviço Projects faz uma chamada ao Users-MS para verificar se o utilizador ainda tem operações disponíveis. Caso o limite diário esteja esgotado, o backend responde com a mensagem “No more daily_operations available”, que é apresentada no frontend através das mensagens uniformizadas (T-02).

Caso o processamento seja iniciado com sucesso, o backend decrementa o número de operações disponíveis. Se o utilizador cancelar, o backend tenta repor as operações associadas a essa execução quando ainda estavam pendentes (pendingAdvancedOps).

No frontend, o número de operações restantes é atualizado automaticamente após cada processamento ou cancelamento, garantindo que o utilizador tem sempre visibilidade do seu saldo de operações. Esta solução cumpre todos os requisitos da tarefa e evita discrepâncias entre frontend e backend.

2.8. T-08 – Remoção de conta e dados (RGPD)

Foi implementada uma ‘Danger Zone’ com um fluxo de remoção de conta. No backend, o Users-MS expõe DELETE /users/:user, que valida a autenticação e desencadeia a remoção de todos os projetos do utilizador no Projects-MS (DELETE /projects/:user), incluindo os recursos associados no storage. Após esse passo, o documento do utilizador é removido no Users-MS, terminando com 204 No Content.

No frontend, a UI confirma a operação com mensagens claras, prevenindo remoções acidentais. Após a conta ser removida, o utilizador é redirecionado automaticamente para a página de login, garantindo que não permanece em áreas protegidas da aplicação.

Toda a operação foi testada para garantir que nenhum rasto dos dados permanece no sistema após a remoção, cumprindo totalmente os requisitos RGPD e garantindo privacidade do utilizador.

2.9. T-09 – Escolha do formato de download

Esta tarefa consistiu em estender a funcionalidade de download para permitir ao utilizador escolher entre diferentes formatos de exportação: ZIP ou PDF, tanto para as imagens originais como para os resultados processados.

O download em ZIP foi corrigido utilizando a biblioteca JSZip, garantindo que o blob é corretamente gerado com type: application/zip e que o nome final corresponde ao esperado:

- projectName.zip para imagens originais;
- projectName_edited.zip para as imagens resultantes do processamento.

O download em PDF foi adicionado com a biblioteca pdf-lib, compilando todas as imagens em páginas distintas, mantendo proporções e suportando tanto PNG como JPEG. Este PDF suporta:

- projectName.pdf (originais)
- projectName_results.pdf (imagens editadas)

A UI foi atualizada com um único dropdown de download, permitindo ao utilizador escolher o formato preferido com uma interação simples e intuitiva. Esta funcionalidade corresponde integralmente ao pedido do requisito e melhora significativamente a usabilidade do sistema.

2.10. T-10 – Cancelar processamento com destaque visual

Embora T-01 implementasse o cancelamento funcional, esta tarefa complementa com foco na experiência do utilizador, garantindo visibilidade clara e controlo sobre operações longas.

Na página do projeto, foi criado um overlay de “Processing” com:

- barra de progresso dinâmica;
- botão Cancelar permanentemente disponível;
- ícone animado e texto de estado;
- desativação da interface subjacente.

Se o processamento ultrapassar os limiares definidos no T-03 (5s para filtros normais, 60s para IA), é mostrado um toast informativo, alertando o utilizador sobre a demora. O cancelamento funciona instantaneamente, graças ao uso de:

- ignoreUpdatesRef (evita eventos tardios do socket);
- cancelProjectProcess (backend encerra o processamento de imediato);
- atualização da sessão para devolver operações utilizadas.

De mencionar que para aumentar a usabilidade e dar maior controlo ao utilizador, o botão de cancelamento está sempre disponível enquanto o processamento decorre, e não apenas após 10s.

Este mecanismo combina visibilidade, controlo e consistência, garantindo que o utilizador nunca fica “preso” num processamento prolongado.

2.11. T-11 — Backups Automáticos Diários

Nesta tarefa foi criado um sistema de backups automáticos diários das bases de dados, permitindo restaurar a informação do utilizador em caso de falha, corrupção ou perda de dados.

Foi introduzido no docker-compose um serviço dedicado denominado mongo-backup, baseado na imagem oficial do MongoDB, colocado na mesma rede dos microserviços, com acesso direto às instâncias users_mongoDB; projects_mongoDB e subscriptions_mongoDB.

No interior do container, um script executa em loop infinito:

- obtém timestamp no formato YYYY-MM-DD-HHMM;
- executa mongodump sobre cada base de dados;
- comprime automaticamente com -gzip;
- guarda os ficheiros na pasta partilhada ./backups/mongo/;
- aguarda 24 horas e repete.

O sistema produz ficheiros como `users-2025-12-01-0252.gz` / `projects-2025-12-01-0252.gz` / `subscriptions-2025-12-01-0252.gz`

A solução foi validada através da inspeção dos ficheiros, visualização dos logs do container e testes de restauração com `mongorestore`, confirmando que os backups estão completos e funcionais. Esta tarefa cumpre integralmente o requisito de robustez e recuperação de dados.

2.12. Gestão de Edições: Undo, Reset e Previews

2.12.1. Solução implementada

A solução passou por reestruturar completamente as funções **Undo** e **Reset**, garantindo que:

- As ferramentas são removidas primeiro no backend, através das mutações apropriadas.
- O estado local (`setTools`) é atualizado apenas depois da confirmação do servidor.
- O preview é executado de forma condicional:
 - gera-se um novo preview se ainda existirem ferramentas,
 - evita-se chamar o backend quando já não há tools, prevenindo erros 500.
- Foram removidas chamadas inválidas a Hooks dentro de funções internas.

Com estas alterações, a coerência entre frontend e backend fica garantida e o comportamento observado pelo utilizador passa a corresponder ao estado real do projeto.

A remoção de ferramentas através de Undo, Reset ou remoção individual reutiliza a seguinte semântica: todas as execuções ativas e outputs são invalidados, garantindo que não existe qualquer possibilidade de o frontend apresentar resultados que já não correspondem à pipeline atual de ferramentas.

2.12.1.1. Undo

- Remove a última ferramenta aplicada no backend e no estado local.
- Se ainda existirem ferramentas após a remoção:
 - é desencadeado um novo preview, atualizado de acordo com a pipeline restante.
- O componente e a lista de tools são atualizados de forma imediata, mantendo a interface responsiva.

Este comportamento corresponde ao **Undo** esperado numa aplicação de edição não-destrutiva baseada em pipeline de ferramentas. De mencionar que se a remoção fizer a pipeline voltar exatamente ao último estado previamente processado (`committedTools`), o backend faz rollback do `committedToken`/`resultsToken` para `prevCommittedToken`, restaurando imediatamente o conjunto de imagens em `results` sem necessidade de novo Apply.

2.12.1.2. Reset

- Remove todas as ferramentas registadas no backend.
- Limpa o estado local de ferramentas.
- Não tenta gerar um novo preview, uma vez que a imagem original já existe; o utilizador volta ao estado inicial do projeto.
- É apresentada uma notificação a indicar que as edições foram removidas e a imagem voltou à versão original.

De mencionar também que o reset completo remove as ferramentas persistidas e evita ficar num estado intermédio em que a partilha fica bloqueada por alterações pendentes.”

2.12.2. Nota sobre o modo results

O modo results mostra o último processamento completo feito pelo backend, através do endpoint: GET /:user/:project/process/url. Isto significa que:

Em geral, Undo/Reset não disparam um novo processamento final. Contudo, o Undo pode restaurar automaticamente o último conjunto de results válido quando a pipeline regressa ao estado previamente processado, através de rollback de committedToken/resultsToken para prevCommittedToken. O Apply continua a ser necessário apenas quando se pretende gerar novos results para uma pipeline diferente do último commit.

Desta forma, evitam-se estados em que results fica vazio ('No results found') após Undo, e mantém-se o editor coerente com o princípio de edição não-destrutiva: ao reverter para um estado já processado, o sistema reapresenta o output já existente em vez de obrigar a recomputar.

O mecanismo de cancelamento opera de forma independente de Undo/Reset. Enquanto Undo e Reset alteram apenas a pipeline de ferramentas, o cancelamento atua exclusivamente sobre execuções em curso, restaurando o estado do modo results através do rollback do committedToken, sem modificar ferramentas persistidas.

3. Novo Caso de Uso: Partilha e Colaboração

3.1. RF50 – Forçar Guardar

3.1.1. Metadados

Descrição: Se existirem alterações não guardadas, o sistema deve solicitar ao utilizador que guarde o projeto antes de continuar com a partilha.

Rationale: Garante que a versão partilhada está atualizada e consistente com o trabalho final do utilizador.

3.1.1.1. Estado global “alterações não guardadas”

Foi criado um contexto global de projeto (ProjectProvider), que expõe, entre outros, os seguintes elementos:

```
hasUnsavedChanges: boolean;  
setHasUnsavedChanges: (dirty: boolean) => void;
```

O ProjectProvider mantém o estado local hasUnsavedChanges e disponibiliza o hook useUnsavedChanges, permitindo que qualquer componente da página do projeto consulte ou altere este estado:

```
export function useUnsavedChanges() {  
  const context = useContext(ProjectContext);  
  return {  
    hasUnsavedChanges: context.hasUnsavedChanges,  
    setHasUnsavedChanges: context.setHasUnsavedChanges,  
  };  
}
```

No próprio ProjectProvider, o estado é limpo quando o Apply termina com sucesso, através de um “sinal” enviado pelo page.tsx:

```
(() => {
  if (preview.waiting === "__APPLY_DONE__") {
    setHasUnsavedChanges(false);
    preview.setWaiting("");
  }
}, [preview.waiting, preview]);
```

3.1.1.2. Sincronização robusta do estado “dirty” e versão do projeto (RNF46)

O backend incrementa o campo version sempre que recebe alterações persistidas relevantes, incluindo a atualização do estado dirty. Isto significa que, ao alternar hasUnsavedChanges, o frontend pode receber um novo X-Project-Version. Se essa versão não for propagada para o estado global do projeto, qualquer operação de escrita seguinte pode enviar um X-Project-Version antigo e originar 409 Conflict (Project version conflict).

Para garantir consistência e evitar conflitos, o ProjectProvider passou a manter a versão do projeto como estado próprio (version), atualizado de duas formas:

- via props: sempre que o projeto chega do backend com project.version, o provider sincroniza o estado local;
- via onSuccess do setProjectDirty: quando a mutation devolve newVersionHeader, o provider atualiza de imediato a versão exposta no contexto.

Com isto, todos os componentes que dependem do project.version (por exemplo, as mutações de Save das tools) passam a usar sempre a versão mais recente conhecida, evitando 409 em cliques consecutivos e garantindo um protocolo de concorrência otimista correto (RNF46).

A atualização é feita expondo no contexto project: { ...project, tools, version }, assegurando que project.version no frontend reflete a versão atualizada pelo próprio sincronismo do estado dirty.

3.1.1.3. Marcação de alterações como “não guardadas” (alterações locais)

Para cumprir o RF50 de forma precisa, o estado hasUnsavedChanges passou a representar apenas alterações locais ainda não persistidas (por exemplo, mexer num slider ou alterar inputs num tool menu sem clicar em “Save”). Ou seja:

- O sistema bloqueia a criação de links apenas enquanto existirem alterações locais por guardar.
- O sistema NÃO bloqueia a partilha só porque existem tools já guardadas no backend mas ainda não aplicadas via Apply.

Esta distinção elimina bloqueios desnecessários e alinha o comportamento com o objetivo do requisito: garantir que o link representa uma versão consistente do estado guardado do projeto.

Na prática, cada componente de tool (Brightness, Contrast, etc.) marca hasUnsavedChanges=true no momento em que o utilizador altera um parâmetro localmente (e.g., onChange do Slider / onChange de Inputs). Quando o utilizador repõe para valores por defeito (Default/Reset local da tool), o estado é limpo.

3.1.1.4. Limpeza do estado (após Save/Default/Reset)

O estado hasUnsavedChanges é limpo quando as alterações locais deixam de existir, nomeadamente:

- quando o utilizador clica em “Save” numa ferramenta (persistindo os parâmetros no backend);
- quando o utilizador seleciona “Default”/reverte os parâmetros locais para o estado guardado;

- quando executa Reset/Undo e a aplicação sincroniza a remoção das ferramentas no backend.

No `ProjectProvider`, existe um mecanismo de “sentinel” (`_APPLY_DONE_`) usado para normalizar o estado da UI após um Apply bem-sucedido, limpando o estado interno de espera (`preview.waiting`). O estado `hasUnsavedChanges` não depende deste sentinel; é controlado pelo ciclo de edição local (alterar parâmetros / guardar / reverter).

3.1.2. Bloqueio da partilha e feedback ao utilizador ->

O componente responsável pela partilha é o `ShareProjectDialog`.

3.1.2.1. Indicador visual no botão “Share”

Enquanto `hasUnsavedChanges` é `true`, é apresentado um ponto laranja junto ao texto do botão:

```
<Button variant="outline">
  <span className="inline-flex items-center gap-1">
    Share
    {hasUnsavedChanges && (
      <span className="inline-block h-2 w-2 rounded-full bg-amber-500" />
    )}
  </span>
</Button>
```

Isto dá ao utilizador um feedback imediato de que existem alterações locais por guardar (por exemplo, parâmetros alterados numa ferramenta sem clicar em “Save”), e que por isso a criação de links de partilha está temporariamente bloqueada.

3.1.2.2. Bloqueio efetivo da criação de links

Ao clicar em “Create link”, o `handleCreate` verifica o estado `hasUnsavedChanges` antes de invocar a mutação `createLink`.

Se existirem alterações não guardadas:

- É apresentado um aviso ao utilizador (toast).
- A função `createLink.mutate` não é chamada, logo não é gerado nenhum link de partilha.

3.1.3. Verificação do critério de aceitação

O fit criterion para o RF50 indica que:

Quando existem alterações não guardadas, o sistema bloqueia a partilha e apresenta um aviso “Guarde o projeto para partilhar”, impedindo a partilha até que o projeto seja guardado.

A implementação cumpre este critério porque:

- `hasUnsavedChanges` passa a `true` sempre que o utilizador altera localmente parâmetros de uma ferramenta (sliders/inputs) sem ainda clicar em “Save”.
 - o estado volta a `false` quando o utilizador guarda a ferramenta, repõe para Default, ou executa Undo/Reset (sincronizado com backend).
 - a criação de links de partilha é bloqueada apenas enquanto `hasUnsavedChanges == true`, garantindo que a partilha nunca é gerada a partir de um estado local ainda não persistido.

O estado volta a false quando o utilizador guarda a ferramenta (persistindo os parâmetros) ou reverte a edição local para o estado guardado. Adicionalmente, quando o projeto é aberto sem permissões de edição (`canEdit == false`), o sistema força `hasUnsavedChanges=false` para evitar inconsistências e impedir estados “dirty” em modo read-only.

- A criação de links de partilha é explicitamente bloqueada enquanto `hasUnsavedChanges == true`, mostrando uma mensagem de erro ao utilizador e não invocando a mutação de criação de link.

Desta forma, a partilha nunca é feita sobre uma versão intermédia do projeto e a versão partilhada está sempre alinhada com o trabalho final do utilizador, conforme o requisito RF5

3.2. RF41 — Edição colaborativa em tempo real (Should)

3.2.1. Mecanismo

Sempre que o servidor persiste uma alteração ao projeto, o Projects-MS publica um evento no RabbitMQ (fila `ws_queue`). O `wsGateway` consome a fila e faz broadcast via `Socket.IO` para a sala `project`, na qual os editores entram ao abrir o projeto (evento `join-project`).

A propagação de alterações usa dois tipos de eventos complementares:

- `project-op` (granular): contém uma operação (ex.: `add/remove/reorder/clear/param update`) e é aplicado diretamente no frontend, atualizando imediatamente o estado local (lista de tools) e a version sem recarregar dados desnecessariamente.

- `project-updated` (fallback/resync): usado quando a alteração não é facilmente aplicada de forma incremental, ou quando se pretende garantir consistência total. Neste caso, o frontend invalida as queries e faz refetch do projeto/imagens/resultados.

Desta forma, a colaboração é “push-based” (sem polling para sincronização do editor), mantendo ainda um mecanismo de resincronização robusto quando necessário.

Recuperação Automática: Além da propagação em tempo real, o sistema integra um mecanismo de recuperação automática de conflitos baseado em versionamento otimista (RNF53). Em caso de conflito de versão (HTTP 409), o frontend invalida o estado (refetch) e apresenta feedback ao utilizador. O controlo de concorrência evita “lost updates” ao exigir `X-Project-Version` em operações de escrita, mas a repetição da operação após conflito não é universal/automática em todas as ações — depende do fluxo.

3.2.2. Efeito prático

Quando um utilizador altera o projeto, os restantes editores na mesma sala recebem imediatamente o evento. Na maioria das operações, a UI atualiza instantaneamente (via **project-op**) apenas o que mudou (tools/ordem/parâmetros) e avança a versão local. Em situações onde é preferível resincronização total, o frontend recebe **project-updated** e faz refetch automático do projeto (e recursos associados), garantindo consistência sem ação manual do utilizador.

Nota: a co-edição aqui é baseada em propagação de operações do estado do projeto (tools/ordem/parâmetros) e não em edição simultânea de texto/caracteres como, por ex, Google Docs.

De mencionar, por fim, que este comportamento foi validado experimentalmente através da simulação de falhas de conectividade (modo Offline / Slow 4G), onde operações locais divergentes foram automaticamente reconciliadas após reconexão, demonstrando suporte efetivo a edição concorrente com recuperação de conflitos.

3.3. RNF65 – Limite de utilizadores num Projeto (Should)

Sobre isto, foi implementado o requisito RNF65, que impõe um limite de 2 colaboradores ativos por projeto quando o projeto pertence a um utilizador não premium e não aplicando restrições quando o dono do projeto é premium. O objetivo é controlar carga e garantir estabilidade, especialmente em modo colaborativo.

A solução foi implementada com base num mecanismo de presence/heartbeat (presença ativa), integrado entre frontend, API Gateway e Projects-MS, garantindo que o limite é aplicado por “editores ativos” (sessões efetivamente a editar naquele momento), e não apenas por utilizadores com acesso histórico.

Durante os testes, foi confirmado que quando o limite de editores ativos é atingido, novas sessões são bloqueadas com HTTP 429. Ao encerrar uma sessão ativa, a presença é libertada e um utilizador previamente bloqueado consegue automaticamente voltar a entrar, sem necessidade de intervenção manual, validando o modelo dinâmico de presença.

3.3.1. Modelo de presença e expiração automática

No Projects-MS foi criado um modelo Presence para registar atividade por utilizador e projeto, com os campos ownerId, projectId, userId e lastSeenAt.

Para evitar “slots presos” (por exemplo, browser fechado sem logout), foi definido um índice TTL que remove automaticamente presenças antigas (expiração após um período de tolerância), assegurando que o número de editores ativos reflete a realidade.

3.3.2. Validação do limite com base no tipo do dono do projeto

A lógica central foi implementada em ensureEditorSlot, que determina o tipo do criador do projeto (owner) através de uma chamada ao Users-MS (/type). Se o dono for premium, a presença é sempre aceite (ilimitado). Se for free, aplica-se o limite 2 (configurável por env), contando apenas presenças recentes dentro de uma janela ativa.

Por robustez, se o Users-MS falhar, o sistema assume um comportamento conservador e trata o owner como free, evitando abrir colaborações ilimitadas por erro.

3.3.3. Middleware de enforcement e códigos de erro

Foi criado o middleware enforcePresenceLimit, que extrai o utilizador “real” (caller) do header X-Caller-Id ou, em fallback, do JWT, e tenta reservar/renovar um slot no projeto.

Quando o limite é atingido, o backend devolve HTTP 429 com uma mensagem e metadados { active, limit }, permitindo ao frontend mostrar um erro explícito e informativo.

3.3.4. Integração nas rotas relevantes (Projects-MS)

O controlo de presença foi aplicado nas operações em que faz sentido contar “colaboradores ativos”:

- Endpoint dedicado de presence POST/DELETE /:user/:project/presence (com verificação de permissões de partilha e de edição).
- Acesso ao projeto (GET /:user/:project): No frontend, o heartbeat de presence só é iniciado quando canEdit == true (owner ou link edit). Em links read, não é feito POST/DELETE de presença e o utilizador não ocupa slot.”

3.3.5. API Gateway: propagação correta de identidade e compatibilidade com modo compartilhado

No API Gateway foram adicionadas rotas de proxy para presence, garantindo que, mesmo em modo compartilhado, o Projects-MS recebe:

- o ownerId real via query (owner=...) quando aplicável;
- o shareId via query (share=...) para validação no Projects-MS;
- o header X-Caller-Id com o utilizador autenticado que está a editar (caller).

3.3.6. Frontend: heartbeat e mensagem ao utilizador (fit criterion)

No frontend, enquanto o editor está aberto em modo edit, é executado um heartbeat a cada 5 segundos para reservar/renovar a presença através do endpoint do API Gateway, e no unmount é feito um DELETE (best-effort) para libertar o slot.

Quando o limite é atingido, o carregamento do projeto em modo compartilhado pode falhar com 429, e a UI apresenta explicitamente: “Limite de colaboradores atingido”, incluindo a contagem active/limit quando disponível — cumprindo o fit criterion do RNF65 (“impede adição e apresenta erro”).

3.3.7. Resultado final

Com esta abordagem, projetos de utilizadores não premium suportam no máximo 2 editores ativos simultâneos, com libertação automática por TTL e feedback claro ao utilizador via HTTP 429 + mensagem dedicada no frontend. Para projetos de owners premium, a colaboração não tem esta restrição.

3.4. RF53 – Acesso por link com permissões (Must).

O sistema distingue permissões read e edit associadas ao link de partilha. O link permite resolver publicamente metadados mínimos. A visualização do projeto completo (imagens + ferramentas) requer autenticação, mesmo em modo read. Em modo edit, a autenticação é igualmente obrigatória antes de abrir o editor em modo de edição.

Justificação: reduzir risco de abuso do link (repartilha não controlada) e permitir auditoria/identificação do utilizador que efetua alterações.

Impacto: o comportamento diverge parcialmente de uma interpretação literal do RF53 (edição diretamente pelo link sem login). Assim, o RF53 é marcado como parcial, embora as permissões e validações no servidor sejam cumpridas.

Em particular, o endpoint que devolve o projeto completo por shareId é protegido (JWT obrigatório), garantindo que um link partilhado não expõe imagens/ferramentas a utilizadores anónimos.

Além da validação no servidor (403 em rotas de escrita), o frontend aplica o mesmo modelo: links read abrem o projeto apenas em modo de visualização dentro do editor (UI consistente, ações de edição desativadas), enquanto links edit permitem ferramentas, previews e Apply.

4. Novo Caso de Uso: Presets

4.1. Requisitos e Objetivo

Nesta fase foi implementado um sistema de presets (“Meus Filtros”) que permite guardar combinações de ajustes e reutilizá-las em qualquer projeto.

A solução cobre os seguintes requisitos: RF39 (criar/guardar presets e disponibilizar na conta), RF66 (aplicar presets), RF67/RF68 (editar e eliminar presets), RF70 (validar autenticação nas operações de gestão) e “listar presets” (predefinidos + personalizados). Do ponto de vista não-funcional, cobre RNF60 (aplicação rápida), RNF63 (persistência), RNF64 (integridade/sem duplicações) e RNF66 (criação apenas quando existem ≥ 2 ferramentas selecionadas).

4.2. Modelo de dados e persistência

No Users-MS, cada preset é armazenado de forma persistente associado ao utilizador (`owner_id`), contendo o nome e uma lista ordenada de ferramentas (`procedure + params`). Esta estrutura garante que o preset reproduz fielmente uma sequência de ajustes em diferentes imagens e projetos (RF39/RF66) e mantém-se disponível após reinício da aplicação (RNF63). Para cumprir integridade (RNF64), o sistema impede duplicação de presets com o mesmo nome para o mesmo utilizador, devolvendo 409 Conflict quando ocorre tentativa de criação/renomeação para um nome já existente.

4.3. API e autenticação (RF70)

A gestão de presets é exposta via API Gateway para o frontend, exigindo autenticação (Bearer token) em todas as operações de criação, edição e remoção (RF70).

Para fechar o comportamento esperado no UC05, foi também introduzida uma regra de **limite máximo de presets por utilizador (4)**. Esta validação é aplicada no backend antes da criação de um novo preset, através de uma contagem de documentos por `owner_id` e retorno de 409 Conflict quando o limite é excedido (“Preset limit (4) reached.”). Em paralelo, a UI foi ajustada para refletir este estado, apresentando placeholder e mensagens específicas (“Limite de 4 presets atingido”) e bloqueando o botão de criação quando o utilizador já atingiu o máximo.

Adicionalmente, para cumprir o requisito de listagem de presets disponíveis (predefinidos + personalizados), o endpoint `GET /users/:user/presets` passou a devolver explicitamente `{ defaultPresets, userPresets }`, e o frontend foi reorganizado em dois separadores (“Predefinidos” e “Meus Filtros”), garantindo que os presets globais apenas podem ser aplicados, enquanto os presets do utilizador suportam o ciclo completo (aplicar/editar/partilhar/eliminar).

4.4. Integração no frontend e desempenho (RNF60/RNF66)

No editor foi adicionada uma interface de “Presets” com dois separadores: “Predefinidos” e “Meus Filtros”. A criação de presets só fica disponível quando o projeto tem pelo menos 2 ferramentas aplicadas (RNF66), evitando configurações vazias. Ao aplicar um preset, o frontend atualiza imediatamente a lista de ferramentas no estado local (resposta visual instantânea), e em seguida sincroniza o backend através da operação de reorder das tools, garantindo uma experiência responsiva e cumprimento de RNF60.

4.5. Edição, remoção e partilha (RF67/RF68/RF39)

Para completar o ciclo de vida de presets, a UI permite renomear e/ou atualizar a sequência de ferramentas (PATCH) (RF67), bem como eliminar presets (DELETE) (RF68). A partilha (RF39) é feita através da criação de um link único: o utilizador gera o shareId e a aplicação copia automaticamente para a área de transferência um URL público. Esse link pode ser aberto numa janela anónima e devolve o preset em formato JSON, provando que o mecanismo de partilha funciona sem dependência de sessão autenticada.

5. Novas Funcionalidades com IA

5.1. Novo Caso de Uso: Assistente IA na edição (UC06 / RF44)

Nesta fase foi implementado um Assistente IA na edição que permite ao utilizador escrever um pedido textual (ex.: “quero um estilo vintage”) e receber sugestões de “receitas” de edição — isto é, sequências de ferramentas já existentes na aplicação (procedures + params). O utilizador pode depois aplicar uma sugestão com um clique, reordenando automaticamente a pipeline de ferramentas do projeto.

5.1.1. Backend (Projects-MS): geração de sugestões

No serviço Projects-MS foi adicionado o endpoint protegido:

POST /:user/:project/assistant/suggest

Este endpoint valida o texto do utilizador e gera uma lista de sugestões com base em keywords normalizadas (ex.: preto&branco, vintage, pop/mais cor, mais claro/escuro). Cada sugestão devolve:

- name (título da sugestão),
- description (explicação curta),
- tools (lista ordenada de tools com procedure e params).

Como detalhe de robustez, foi incluída uma normalização de texto (lowercase + remoção de diacríticos) para tornar a deteção de intenções mais tolerante a variações linguísticas, e um filtro para evitar devolver sugestões “iguais” às ferramentas atuais.

O endpoint respeita o modelo de permissões do modo partilhado e a concorrência otimista por versão, usando os middlewares:

- checkSharePermission + requireEditPermission (impede sugestões/aplicação em links sem permissão de edição),
- requireProjectVersion (garante consistência do estado do projeto).

5.1.1.1. Sugestões adaptativas ao conteúdo da imagem (features + imgId)

Durante a validação do Assistente IA na edição foi identificado um problema: as sugestões eram geradas apenas a partir do texto do utilizador, ignorando o conteúdo da imagem. Na prática, isto fazia com que o mesmo prompt (ex.: “mais cor”) devolvesse receitas muito semelhantes independentemente da imagem selecionada, levando a resultados inconsistentes (ex.: aumentar demasiado o contraste em imagens já contrastadas).

Para corrigir este comportamento, o endpoint `POST /:user/:project/assistant/suggest` foi estendido para receber também o `imgId` e passar a gerar sugestões dependentes da imagem “corrente”. O fluxo implementado no Projects-MS passou a ser:

- Seleção da imagem alvo: o backend tenta resolver `imgId` para uma entrada real em `project.imgs`. Se não existir, faz fallback para a primeira imagem do projeto;
- Download do ficheiro real: obtém-se uma URL temporária do MinIO (`get_image_docker`) e descarrega-se o conteúdo como `arraybuffer` (Axios), convertendo para `Buffer`;
- Extração de **features** com sharp: são obtidos `metadata()` e `stats()` para calcular dimensões, brilho médio e contraste;
- Geração de sugestões adaptativas: a função `buildSuggestions(message, currentTools, features)` calibra automaticamente os valores de `brightness`, `contrastFactor` e `saturationFactor` com base nessas métricas, usando **clamps** para evitar overcorrection.

A extração de features foi implementada em `utils/imageFeatures` e devolve:

- `width` e `height` (dimensões);
- `meanBrightness` (0..255), calculado por luminância ponderada ($0.2126R + 0.7152G + 0.0722B$);
- `contrast` (aprox.), estimado pela média do desvio padrão (`stdev`) dos canais RGB.

Com estas alterações, o mesmo pedido textual (“mais cor”, “vintage”, “preto e branco”, etc.) passa a produzir receitas diferentes para imagens diferentes. Por exemplo, imagens com contraste elevado recebem um `contrastFactor` mais conservador, enquanto imagens mais “lavadas” recebem um reforço maior. Isto aumentou a consistência visual e eliminou discrepâncias observadas entre imagens distintas com o mesmo prompt.

5.1.1.2. RF61 — Preview visual das sugestões (pré-visualização por “receita”)

Após obter as sugestões do Assistente IA, foi acrescentado suporte para apresentar um preview visual por sugestão, permitindo ao utilizador comparar rapidamente o efeito esperado antes de “Aplicar”. Para manter o sistema responsivo e evitar acionar o pipeline completo (RabbitMQ + Tools-MS) apenas para pré-visualizações, o preview é gerado localmente no Projects-MS, aplicando apenas tools não-IA (ex.: `brightness`, `contrast`, `saturation`).

No backend foram introduzidas as funções auxiliares:

- `applyToolsPreview(inputBuffer, tools)`: usa sharp para aplicar sequencialmente as tools suportadas:
 - `brightness`: `sharp(...).modulate({ brightness })`
 - `saturation`: `sharp(...).modulate({ saturation })`
 - `contrast`: implementado via `linear(a,b)` (com $a=\text{contrastFactor}$ e $b=128*(1-\text{contrastFactor})$), dado que o sharp não disponibiliza um `contrastFactor` direto;
- `uploadAssistantPreview(ownerId, projectId, outBuffer)`: faz upload do buffer gerado para o bucket já existente `preview` (MinIO), e resolve uma URL temporária com `get_image_host`.

Assim, o endpoint `POST /:user/:project/assistant/suggest` passou a devolver, para cada sugestão, além de `{name, description, tools}`, também:

- `previewUrl`: URL temporária do MinIO para renderização direta no frontend;
- `previewKey`: chave do objeto no bucket `preview` (útil para limpeza futura/diagnóstico).

Do lado do frontend, o diálogo `AiAssistantDialog` foi atualizado para renderizar a imagem quando `previewUrl` existe, mostrando-a dentro do cartão da sugestão (com `object-contain` e altura limitada), o que melhora significativamente a usabilidade: o utilizador pode ver o efeito esperado (ex.: “Pop/Mais cor”, “Realce de cor”, “Luz & Cor”) sem ter de aplicar/reverter ferramentas manualmente.

Este mecanismo é tolerante a falhas: caso a geração/upload do preview falhe para uma sugestão, a sugestão continua a ser devolvida (com `previewUrl=null`), garantindo que o Assistente IA não fica indisponível devido a problemas pontuais no preview.

5.1.2. API Gateway: encaminhamento seguro para o Projects-MS

No API Gateway foi criado o endpoint correspondente que faz proxy para o Projects-MS, mantendo o mesmo path:

```
POST /projects/:user/:project/assistant/suggest
```

Este encaminhamento preserva autenticação e controlo de versão ao reenviar:

- Authorization (JWT),
- X-Project-Version (versão esperada),
- X-Caller-Id (utilizador que iniciou a ação),
- share/owner quando aplicável (modo partilhado).

5.1.3. Frontend: UI do assistente e edição de sugestões

No frontend foi criado o componente `AiAssistantDialog`, exposto na barra de ações do editor através do botão “IA na edição”. Este diálogo permite ao utilizador introduzir um pedido textual, enviar o prompt para o backend e visualizar a lista de sugestões devolvidas pelo serviço de Assistente IA.

Cada sugestão é apresentada com o respetivo nome, descrição e sequência de ferramentas propostas. Ao receber as sugestões, o frontend converte a lista de tools sugeridas para o formato interno do projeto, contendo os campos `procedure`, `params` e `position`.

Para cumprir o requisito RF63, foi introduzida uma etapa intermédia de edição manual das sugestões através do componente `AiSuggestionEditor`. A interface suporta dois modos de operação: um modo de lista, onde são mostradas todas as sugestões com as opções “Editar” e “Aplicar”, e um modo de editor, onde a sugestão selecionada é apresentada num formulário dedicado que permite ajustar manualmente os parâmetros de cada ferramenta antes da aplicação.

O `AiSuggestionEditor` apresenta, para cada ferramenta da sugestão, um bloco com o nome do `procedure` e um conjunto de campos de entrada correspondentes aos parâmetros configuráveis (`params`). Como os valores provenientes dos inputs HTML são devolvidos sob a forma de strings, o frontend converte novamente estes valores para o tipo numérico adequado sempre que necessário, evitando falhas de execução nas ferramentas de processamento.

A aplicação de uma sugestão, original ou editada, reutiliza a operação de reordenação já existente no sistema. Antes de enviar os dados para o backend, o frontend remove quaisquer identificadores temporários (`_id`) e recalcula o campo `position` de forma determinística, garantindo compatibilidade com o payload esperado pelo serviço Projects-MS. O pedido é então submetido através do endpoint de `reorder`, atualizando a sequência de ferramentas do projeto de forma consistente.

5.1.4. Integração na página do projeto

O AiAssistantDialog foi integrado em /dashboard/[pid], recebendo os dados necessários ao funcionamento em modo normal ou partilhado:

- uid, pid, token
- ownerId e shareId (quando o editor é aberto por link)
- projectVersion e currentTools (estado atual do projeto).

5.1.5. Validação (fit criteria)

A funcionalidade foi validada de ponta-a-ponta: criar projeto, adicionar tools manuais, pedir sugestão textual e aplicar. Após “Aplicar”, a pipeline de ferramentas é atualizada de acordo com a sugestão, refletindo imediatamente a nova ordem no editor, e ficando persistida para processamento final.

5.2. RF40 – Generative Expand (expand_ai)

Nesta fase foi implementada a ferramenta Generative Expand (procedure expand_ai), que permite expandir o canvas de uma imagem adicionando margem em torno do conteúdo original. O objetivo é criar espaço adicional para posterior edição (ex.: recorte, composição ou preparação para watermark), mantendo continuidade visual entre a imagem original e as zonas expandidas. Em fundos texturados o resultado é visualmente coerente; em objetos isolados podem surgir artefactos, dado que o motor generativo utilizado em ambiente local é apenas um mock heurístico.

A solução foi integrada de forma consistente com a arquitetura distribuída existente, recorrendo a:

- microserviço dedicado em Tools;
- comunicação assíncrona via RabbitMQ;
- integração completa no editor, com suporte a preview e apply.

5.2.1. Parâmetros suportados e comportamento

A ferramenta aceita parâmetros que definem a dimensão da expansão e o método de preenchimento das áreas novas:

- percent – percentagem de expansão aplicada proporcionalmente em todas as direções;
- mode – modo de preenchimento das zonas novas:
 - reflect – espelha as bordas da imagem;
 - edge – replica o pixel de borda;
 - solid – preenche com uma cor sólida;
- color – utilizado apenas no modo solid, no formato #RRGGBB.

O processamento base foi implementado com Pillow (PIL), criando o novo canvas e preservando integralmente a área original.

No modo generative são ainda aceites: prompt; negativePrompt (opcional); seed (opcional); steps; guidance.

O frontend apenas envia estes campos quando o modo selecionado é generative. No serviço mock estes parâmetros são apenas registados para diagnóstico, não influenciando o resultado final.

5.2.2. Integração no backend (Tools + RabbitMQ)

A ferramenta foi implementada como microserviço em Tools/expand_ai/ (expand_ai.py, Dockerfile, requirements.txt).

Durante o processamento:

- é calculado o padding;
- no modo generative é criado um canvas expandido e uma máscara que marca as zonas novas;
- canvas e máscara são enviados por HTTP para o endpoint /outpaint, definido por GEN_EXPAND_URL.

A comunicação assíncrona mantém-se baseada em RabbitMQ, lendo da fila expand_ai_queue e publicando resultados na project_queue.

5.2.3. Serviço generative-mock para testes locais

Devido à impossibilidade de executar Stable Diffusion em Apple Silicon (arm64), foi criado um serviço generative-mock que simula o contrato e o efeito de preenchimento das margens, sem geração semântica real.

Este serviço:

- expõe GET /sdapi/v1/samplers como healthcheck;
- expõe POST /outpaint, recebendo image e mask e executando preenchimento das áreas novas através de inpainting com OpenCV (cv2.inpaint, método Telea), onde 255 na máscara representa a área a gerar.

O serviço é definido no docker-compose.yaml como generative-mock, exposto na porta 7860 e utilizado pelo expand_ai_tool através da variável GEN_EXPAND_URL. São emitidos logs com dimensões do canvas, presença de máscara e rácio de área preenchida, permitindo validar que apenas as margens são alteradas.

5.2.4. Alterações de infraestrutura (docker-compose)

Foram introduzidos dois serviços: **generative-mock & expand_ai_tool**

O expand_ai_tool depende explicitamente do RabbitMQ e do generative-mock via healthchecks, garantindo que só inicia após ambos estarem operacionais.

5.2.5. Integração no frontend (Toolbar + Preview/Apply)

A ferramenta foi adicionada à Toolbar através do componente ExpandAITool, permitindo configurar:

- percentagem de expansão;
- modo (reflect, edge, solid, generative);
- parâmetros avançados no modo generative.

O frontend constrói dinamicamente os parâmetros a enviar, respeitando o contrato do backend.

5.2.6. Validação (fit criteria)

O requisito RF40 – Generative Expand encontra-se implementado ao nível funcional e arquitetural. Apesar de o motor utilizado em ambiente local não corresponder a um modelo de difusão real, a solução garante:

- integração completa na arquitetura distribuída;
- cumprimento do contrato /outpaint (canvas + máscara);
- existência de geração visível nas zonas expandidas, através de inpainting heurístico;
- possibilidade de substituição transparente por um motor generativo real, sem alterações ao frontend ou ao pipeline.

Desta forma, a funcionalidade encontra-se validada enquanto proof-of-concept operacional do fluxo de generative expand, ficando a qualidade semântica da geração dependente apenas da troca do serviço mock por um modelo de difusão real.

6. Melhorias implementadas ao projeto

6.1. Melhorias no processamento de projetos

Durante o desenvolvimento da aplicação, foi identificada uma limitação significativa no comportamento do processamento de imagens. O sistema original assumia que qualquer ação de “Apply/Process” implicava sempre repercutir **todas as ferramentas da pipeline sobre todas as imagens** — mesmo quando o utilizador não tinha feito alterações relevantes, e sobretudo mesmo quando já tinha pago anteriormente pelas ferramentas de IA envolvidas.

Este comportamento tinha várias consequências negativas:

- processamento desnecessário, mais lento e menos eficiente;
- consumo indevido de operações de IA ao repetir ferramentas avançadas já processadas;
- inconsistências no cancelamento e no reembolso de créditos;
- dificuldade em controlar corretamente o número de ferramentas avançadas realmente utilizadas.

6.2. Reestruturação da contagem de ferramentas avançadas

Para corrigir este problema, foi introduzida uma nova lógica centrada na ideia de que **só conta — e só consome operações — aquilo que é novo desde o último processamento válido**.

Foi implementado um mecanismo que distingue:

1. **Ferramentas avançadas já aplicadas e pagas** Guardadas no campo: `chargedAdvancedTools`.
2. **Ferramentas avançadas adicionadas desde o último processamento** Calculadas como: $\text{newTools} = \text{totalAdvancedTools} - \text{chargedAdvancedTools}$.
3. **Operações efetivamente necessárias nesta execução** $\text{adv_ops} = \text{newTools} * \text{número de imagens}$.

Assim, só estes `adv_ops` são enviados para o `users-ms`, e apenas quando existe realmente algo novo a cobrar. O utilizador passo então a só “pagar” **na primeira vez que usa cada ferramenta de IA**.

6.3. Novo fluxo de processamento

6.3.1. Utilizador adiciona ferramentas (IA e não-IA)

Sem qualquer custo imediato.

6.3.2. Ao clicar em “Apply”, o sistema verifica

- quantas ferramentas avançadas existem;
- quantas já foram pagas anteriormente;
- quantas são novas.

E assim, apenas as novas são cobradas.

6.3.3. A pipeline completa é aplicada

Esta é aplicada para garantir consistência dos resultados, mas o custo só incide sobre ferramentas avançadas novas.

6.3.4. No caso de cancelamento

A lógica foi reescrita para garantir que apenas as operações reservadas para aquela execução são devolvidas.

6.4. Cancelamentos e reembolsos de operações

Originalmente, o sistema não reembolsava corretamente o número de operações gastas na execução corrente, resultando em perda permanente de quota diária sempre que o processamento era interrompido.

A nova chamada ao users-ms (POST /:user/process/refund/:advanced_tools) introduz uma lógica precisa e robusta:

- identifica apenas as operações gastas na execução atual;
- subtrai exatamente esse valor ao registo do dia;
- impede que o número fique negativo;
- garante que utilizadores premium não são afetados por quotas.

Cada execução mantém registo separado das operações avançadas pendentes. No cancelamento, apenas as operações associadas ao token cancelado são devolvidas, impedindo reembolsos indevidos de execuções anteriores já concluídas e o contador de operações passa assim a refletir o estado real.

6.5. Benefícios da nova abordagem

A aplicação torna-se mais:

- **eficiente**: não repete cobranças e evita overbilling;
- **previsível**: o utilizador só paga quando adiciona novas ferramentas de IA;
- **robusta**: integração consistente entre microserviços mesmo em erros ou cancelamentos;
- **escalável**: pipelines maiores não prejudicam o custo final para o utilizador.

6.6. As ferramentas não-IA seguem esta lógica?

6.6.1. No que diz respeito a custos

Ferramentas não-IA (ex.: brightness, contrast) => **não consomem operações** => **não entram no cálculo de cobrança** => logo não participam nos campos:

- chargedAdvancedTools
- newTools
- adv_ops
- refund

6.6.2. No que diz respeito ao comportamento da pipeline

Seguem exatamente a mesma lógica de execução:

- são reaplicadas em todos os processamentos;
- respeitam a ordem definida pelo utilizador;
- interferem no resultado final.

6.7. Modo escuro (RNF59)

Foi implementado o requisito RNF59 — Modo Escuro — permitindo ao utilizador alternar entre tema claro e tema escuro a partir das definições da aplicação. Para isso, foi introduzido um ThemeProvider global que guarda a preferência no localStorage (persistência entre sessões) e aplica a classe dark ao elemento root do documento, ativando as variantes dark do Tailwind/Shadcn UI em toda a aplicação.

No frontend, foi adicionada uma secção “Aparência” na página de definições de conta, com um botão que ativa/desativa o modo escuro e indica o estado atual (Ativo/Inativo). Esta abordagem garante uma alteração imediata do tema sem refresh, mantendo consistência visual em todos os componentes.

7. Design Decisions

Nesta secção, apresenta-se o mapeamento detalhado entre os requisitos funcionais e não funcionais e os componentes da arquitetura do sistema. Esta alocação visa demonstrar como a arquitetura de microsserviços adotada suporta as novas funcionalidades da Fase 2, garantindo a separação de responsabilidades e a modularidade do sistema.

7.1. Padrão Arquitetural

A solução adota como padrão arquitetural principal uma arquitetura de Microsserviços, complementada por um API Gateway e por componentes especializados para comunicação em tempo real. O sistema encontra-se estruturado em serviços independentes, tais como Projects-MS, Users-MS, Tools-MS e WebSocket Gateway, cada um responsável por um conjunto coeso de funcionalidades e com dados próprios.

A identificação deste padrão é suportada por várias evidências arquiteturais. Em primeiro lugar, cada microsserviço possui a sua própria instância de base de dados, garantindo isolamento de dados e autonomia na persistência. Em segundo lugar, a comunicação entre serviços é heterogénea, recorrendo a REST via HTTP para operações síncronas de negócio, mediadas pelo API Gateway, e a AMQP via RabbitMQ para processamento assíncrono e intensivo, particularmente associado a funcionalidades baseadas em IA. Adicionalmente, cada serviço é executado em contentores isolados, com o seu próprio ciclo de vida e stack tecnológica, reforçando o isolamento de processos. A existência de um ponto único de entrada, assegurado por um Reverse Proxy e por um API Gateway, constitui igualmente uma característica típica deste padrão arquitetural.

A adoção de uma arquitetura de microsserviços influencia diretamente os requisitos não funcionais críticos do sistema. Em termos de escalabilidade, permite escalar de forma independente componentes com maior carga, como o WebSocket Gateway e o Tools-MS, particularmente relevantes no contexto de colaboração em tempo real e processamento intensivo associado à IA. Relativamente ao desempenho, a execução assíncrona e a comunicação em tempo real reduzem a latência percebida pelo utilizador.

Em síntese, o padrão arquitetural adotado revela-se adequado aos requisitos do sistema, suportando de forma eficaz atributos de qualidade como escalabilidade, desempenho, robustez e consistência, à custa de um aumento controlado da complexidade de gestão e comunicação entre serviços, considerado aceitável face aos benefícios obtidos.

7.2. Tabela de alocação funcional dos requisitos

ID Requisito	Descrição	Componente Alo- cado	Novo componente necessário?	Justificação
RF41	Edição colaborativa em tempo real	Frontend, API Gateway, Projects-MS, WebSocket Gateway	Não	O uso de um wsGateway isolado permite desacoplar a lógica de negócio do transporte de mensagens, promovendo modularidade e escalabilidade independente.
RF40	Ferramenta Generative Expand	Frontend, API Gateway, Projects-MS, Tools-MS	Sim	A implementação como microserviço isolado garante que dependências pesadas de IA não impactam a gestão de projetos, assegurando baixo acoplamento e modularidade.
RF53	Acesso por Link	API Gateway, Projects-MS	Não	Controlo de acesso por link implementado no Projects-MS, garantindo coesão entre persistência do projeto e validação de permissões.
RF44	Assistente IA	Frontend, API Gateway, Projects-MS	Não	Manter geração no Projects-MS porque precisa de project.tools, project.imgs e download do MinIO + features/preview local.

ID Requisito	Descrição	Componente Alo- cado	Novo componente necessário?	Justificação
RF50	Forçar Guardar	Frontend, API Ga- teway, Projects-MS	Não	Frontend bloqueia por estado lo- cal; backend ga- rante consistência com project.dirty e requireProjectVer- sion.
RF39	Criar e gerir filtros personalizados	Frontend, API Ga- teway, Users-MS	Não	Presets associados ao utilizador são geridos no Users- -MS, responsável pelas preferências e dados pessoais.
RF70	Verificar autentica- ção do utilizado	Frontend, API Ga- teway, Users-MS	Não	Validação de JWT centralizada no API Gateway, garan- tindo autenticação antes do encami- nhamento para os microserviços.

Tabela 2: Tabela de alocação Funcional de requisitos

7.3. Tabela de alocação dos requisitos não funcionais

ID RNF	Atributo de Qualidade	Descrição	Componente(s) Relacionado(s)	Tática Aplicada / Proposta
RNF65	Disponibilidade/Performance	Limitar colaboradores em simultâneos em contas Free	API Gateway, Projects-MS	Implementação de padrão Heartbeat com TTL para limitar sessões ativas (Max 2).
RNF59	Usabilidade	Modo escuro	Frontend	Utilização de um ThemeProvider global com persistência em LocalStorage para garantir a consistência visual entre sessões.
RNF60	Desempenho	Tempo de aplicação de preset	Frontend, API Gateway, Tools-MS	Execução assíncrona com feedback em tempo real via Web-Sockets para reduzir a latência percebida.
RNF46	Consistência	Verificação de Alterações	Frontend, API Gateway, Projects-MS	Uso do header X-Project-Version para detetar conflitos de escrita e prevenir lost updates em edições simultâneas.
RNF63	Persistência	Armazenamento persistente de presets	Users-MS, Projects-MS	Armazenamento centralizado e persistente, garantindo que os presets sobrevivem a falhas do serviço ou reinícios da aplicação.
RNF64	Integridade	Consistência de dados de presets	Users-MS	Validação de esquema ao nível da base de dados e API para impedir a duplicação de nomes de presets para o mesmo utilizador.
RNF66	Robustez	Disponibilidade da criação de presets	Frontend, API Gateway, Users-MS	Implementação de lógica de controlo no frontend e backend que impede a criação de presets com menos de 2 ferramentas, garantindo o estado válido do sistema.

Tabela 3: Tabela de alocação dos requisitos não funcionais

8. Building Block View

Este capítulo apresenta a *textit{Building Block View}* do sistema *textit{pictuRAS}*, recorrendo a um diagrama de componentes para sintetizar a arquitetura atual implementada na Fase 2. O objetivo principal desta vista é evidenciar, a um nível estrutural, os blocos de software que compõem a solução e as dependências entre eles, tornando explícita a separação entre: (i) a camada de entrada e entrega (*textit{edge/entry}*), (ii) os componentes de interface e gateways, (iii) os microserviços nucleares (*textit{core services}*), (iv) a infraestrutura e persistência de dados, e (v) o ecossistema de ferramentas e *textit{workers}* responsáveis por processamento assíncrono.

O diagrama foi construído para corresponder diretamente à implementação corrente e às principais decisões de arquitetura introduzidas nesta fase: exposição controlada através de *textit{reverse proxy}*, comunicação síncrona por REST entre serviços de negócio, comunicação assíncrona por AMQP para execução de ferramentas e disseminação de eventos, e persistência independente por microserviço (MongoDB por domínio). Adicionalmente, a vista inclui os serviços de suporte (storage compatível com S3, backups automáticos e serviço *textit{mock}* para testes locais) que garantem a reprodutibilidade e robustez do sistema durante desenvolvimento e validação.

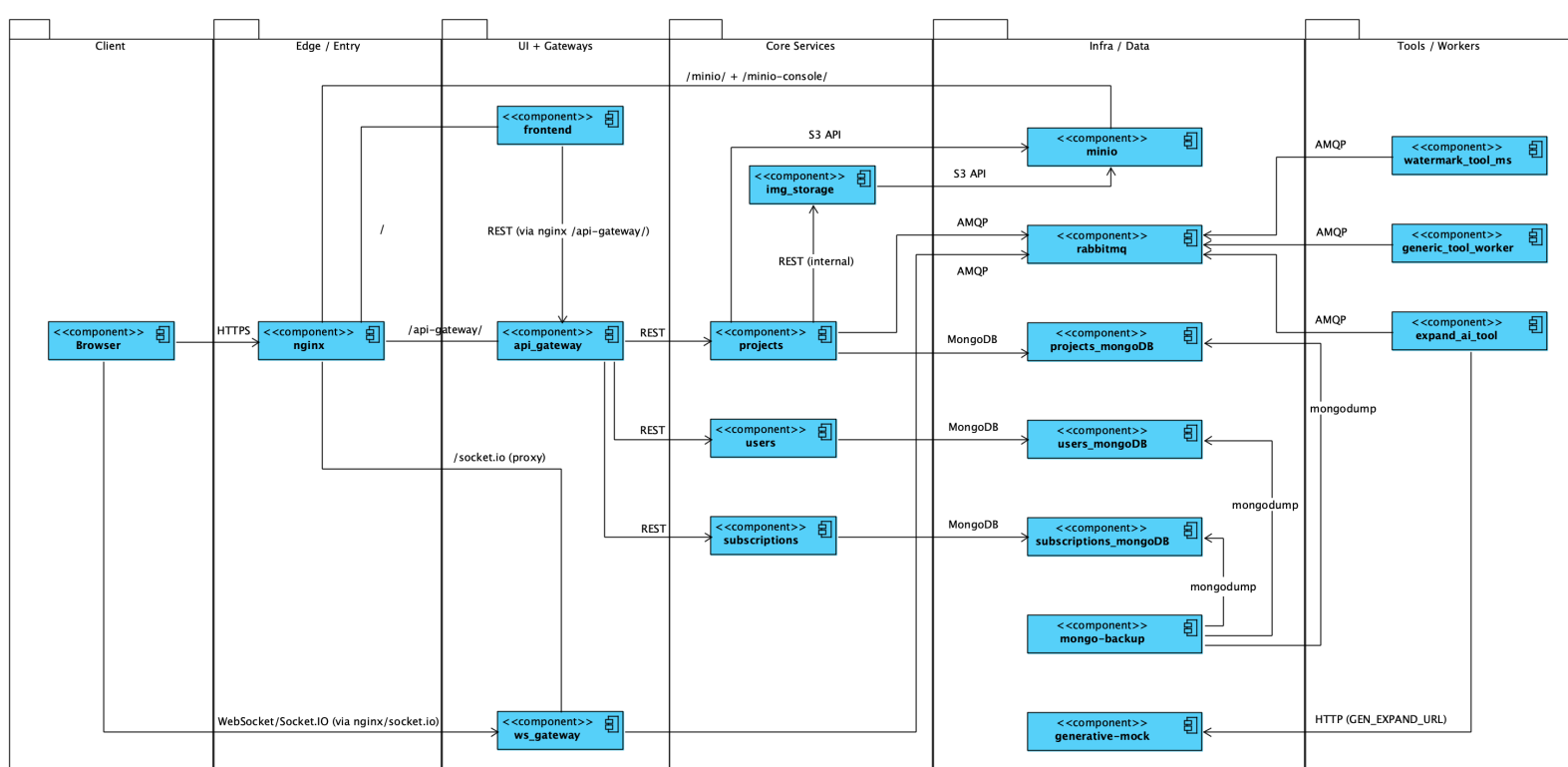


Figura 1: Diagrama de Componentes referente ao *Building Block View*.

A Figura 1 organiza os componentes em seis zonas, refletindo responsabilidades e fronteiras de confiança:

- **Client** — representa o consumidor final do sistema. O **Browser** é o ponto de entrada do utilizador.
- **Edge / Entry** — o **nginx** funciona como *textit{reverse proxy}* e ponto único de exposição, consolidando rotas e garantindo comunicação cifrada com o cliente.
- **UI + Gateways** — inclui o **frontend** (interface web), o **api_gateway** (orquestração/encaminhamento de REST para os microserviços) e o **ws_gateway** (gateway de tempo real via Socket.IO).
- **Core Services** — microserviços de domínio: **projects**, **users** e **subscriptions**, além de **img_storage** para operações internas relacionadas com ficheiros/imagens.
- **Infra / Data** — componentes de infraestrutura e persistência: **rabbitmq** (mensageria AMQP), **minio** (storage compatível com S3), bases de dados MongoDB por serviço (**projects_mongoDB**, **users_mongoDB**, **subscriptions_mongoDB**), o serviço de **mongo-backup** (backups periódicos) e o **generative-mock** (suporte a testes locais).

- **Tools / Workers** — serviços responsáveis por processamento assíncrono (por exemplo **watermark_tool_ms**, **expand_ai_tool** e um `textit{worker}` genérico representativo do padrão comum).

Do ponto de vista dos fluxos, a entrada do utilizador é feita através de HTTPS no **nginx**, que encaminha:

- (1) requisições do **frontend** (rota `"/`),
 - (2) chamadas de API para o **api_gateway** (prefixo `"/api-gateway/"`),
 - (3) ligações WebSocket/Socket.IO para o **ws_gateway** (rota `"/socket.io"`),
- e ainda (4) acesso aos endpoints do **minio** e da consola (`"/minio/"` e `"/minio-console/"`).

Embora o diagrama apresente HTTPS entre **Browser** e **nginx**, é relevante notar que, no ambiente de execução local, este serviço pode estar mapeado numa porta externa diferente (mapeamento de porta no container), mantendo-se a semântica de TLS/HTTPS do ponto de vista do cliente.

A comunicação síncrona de negócio é efetuada por REST: o **frontend** consome a API via **nginx** (isto é, "REST via `nginx "/api-gateway/"`"), e o **api_gateway** encaminha pedidos para os microserviços **projects**, **users** e **subscriptions**. Em termos de segurança e consistência, o gateway preserva a autenticação (JWT) e propaga o contexto necessário a funcionalidades avançadas como partilha/colaboração, garantindo que o serviço de domínio aplica as regras de autorização e concorrência.

A comunicação assíncrona está centralizada em **rabbitmq** (AMQP). Nesta arquitetura:

- os serviços de domínio (com destaque para **projects**) publicam pedidos de processamento e eventos relevantes;
- os componentes em **Tools / Workers** consomem pedidos, executam a transformação (incluindo ferramentas de IA), e publicam resultados/estado de volta no broker;
- o **ws_gateway** consome eventos e emite atualizações em tempo real via Socket.IO para os clientes ligados.

Este desenho desacopla o processamento pesado do caminho síncrono REST, melhora escalabilidade e permite que operações longas sejam acompanhadas/canceladas sem bloquear a interface.

Relativamente a dados e ficheiros, a persistência é segregada por domínio: cada microserviço mantém a sua base de dados MongoDB (setas "MongoDB"), evitando acoplamento entre esquemas. O armazenamento de imagens é feito em **minio** através de uma API compatível com S3 (setas "S3 API"). É importante explicitar que o serviço **img_storage** representa um componente interno usado para operações auxiliares sobre ficheiros, sendo consumido pelo serviço **projects** via "REST (internal)". O serviço **mongo-backup** automatiza backups das bases de dados (setas "mongodump"), reforçando robustez e recuperação.

Finalmente, a ferramenta **expand_ai_tool** integra com um endpoint externo configurável, representado por **generative-mock** no ambiente local (seta "HTTP (GEN_EXPAND_URL)"). Esta escolha permite validar o pipeline de IA fim-a-fim sem dependência de infraestrutura pesada, mantendo o comportamento contratual do sistema (requests e respostas) durante testes.

Em síntese, a Figura 1 evidencia:

- (i) entrada única e segura via **nginx**,
- (ii) separação entre tráfego síncrono (REST) e assíncrono (AMQP),
- (iii) isolamento de dados por microserviço
- (iv) integração de processamento extensível através de `textit{workers}` e tools, suportando as novas funcionalidades da Fase 2 (incluindo colaboração e operações avançadas) de forma modular e escalável.

9. Runtime View - Colaboração (UC07)

Esta secção apresenta a Runtime View do UC07 (colaboração), ilustrando o fluxo completo desde a abertura de um link de partilha por um convidado até à edição concorrente do mesmo projeto por múltiplas sessões. O objetivo é evidenciar a integração entre o caminho síncrono (REST via API Gateway) e o caminho assíncrono de tempo real (RabbitMQ + ws_gateway/Socket.IO), bem como a estratégia de concorrência adotada baseada em versionamento otimista (X-Project-Version) para evitar lost updates em alterações simultâneas.

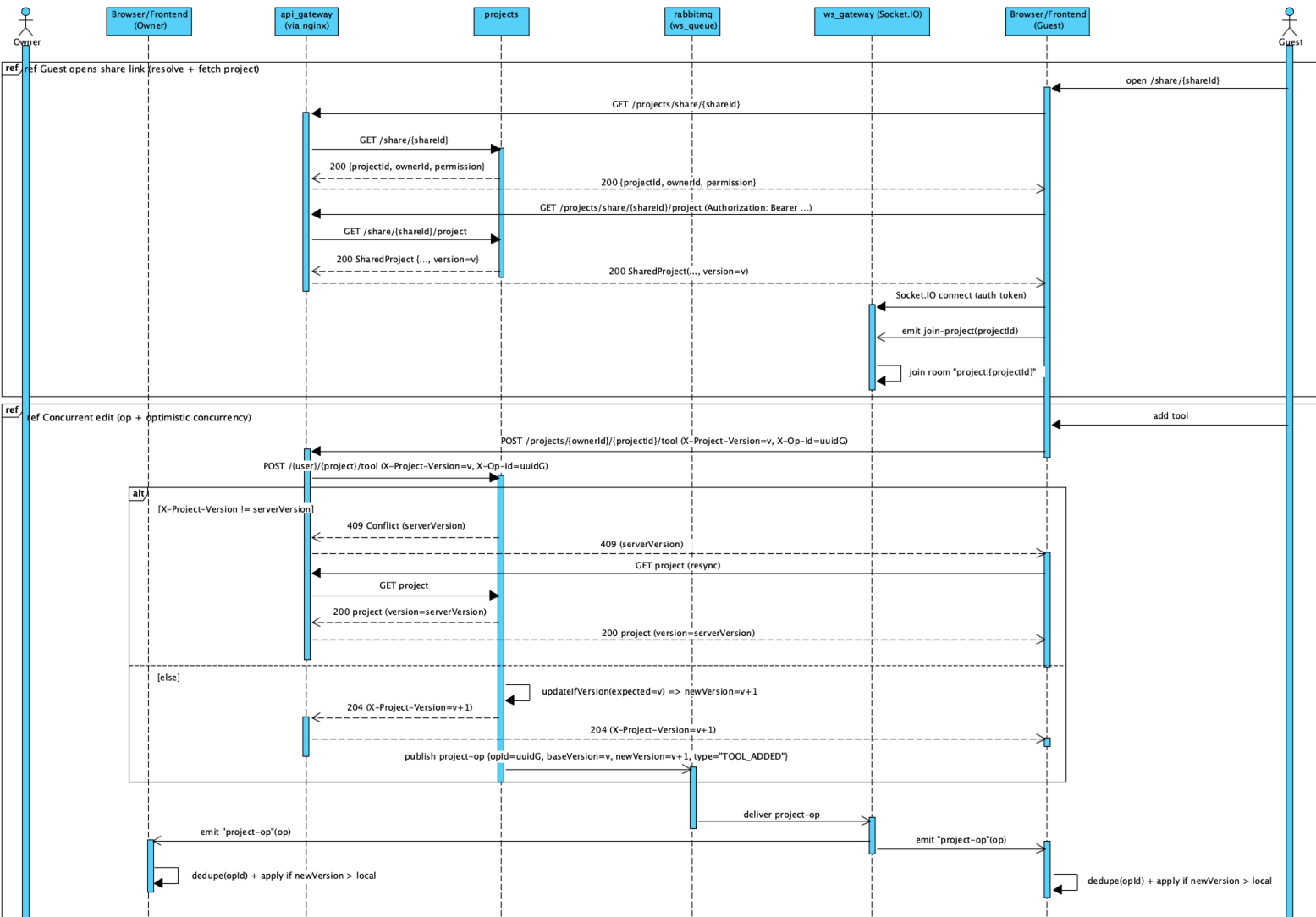


Figura 2: Diagrama de Sequência - Colaboração

O diagrama divide-se em dois momentos. No primeiro, o convidado abre o URL público /share/{shareId} e o frontend resolve o link através do endpoint de partilha, obtendo metadados mínimos (ex.: projectId, ownerId, permission). Em seguida, após autenticação, o frontend carrega o projeto completo através do endpoint autenticado do projeto partilhado. Este passo estabelece o bootstrap do editor em modo partilhado, garantindo que o acesso ao conteúdo do projeto só ocorre com token válido e que a permissão (read/edit) é conhecida antes de iniciar a sessão de edição.

No segundo momento, a colaboração em tempo real é garantida por um mecanismo push-based. Quando um utilizador (owner ou convidado com permissão edit) executa uma operação de escrita (ex.: adicionar ferramenta), o frontend envia o pedido REST incluindo X-Project-Version e um identificador de operação (X-Op-Id). O Projects-MS valida a versão esperada e, se houver divergência, rejeita com 409 Conflict, permitindo ao frontend ressincronizar (refetch) e evitar perda silenciosa de alterações. Quando a versão coincide, a operação é aceite, a versão do projeto avança e o servidor devolve a nova versão, que passa a ser a referência para operações seguintes.

Após persistir a alteração, o Projects-MS publica um evento na fila ws_queue (RabbitMQ). O ws_gateway consome essa fila e emite eventos Socket.IO para a sala do projeto (ex.: project:{projectId}), na qual os editores entram ao abrir o projeto (evento join-project). Para atualizações eficientes, a propagação é feita preferencialmente via eventos do tipo project-op, que transportam apenas a operação (granular) e permitem atualizar o estado local (tools/ordem/parâmetros) sem recarregar o projeto inteiro; quando não é viável aplicar incrementalmente, existe um evento project-updated como fallback para forçar ressincronização via refetch. Este desenho reduz tráfego e mantém a UI responsiva durante edição simultânea.

Finalmente, para robustez, o frontend aplica deduplicação e validação por versão: operações já vistas (mesmo opId) são ignoradas e apenas eventos com versão mais recente do que a local são aplicados. Assim, mesmo com reentregas do broker, latência variável ou múltiplos clientes, o estado converge e o risco de inconsistências é mitigado pelo protocolo de versão e pelo mecanismo de ressincronização quando necessário.

10. Runtime View - Cancelamento

Esta secção apresenta a `textit{Runtime View}` relativa ao cancelamento de um processamento em curso, correspondendo às tarefas T-01 e T-10. O objetivo desta vista é demonstrar, de forma dinâmica, como o sistema lida com pedidos de cancelamento durante a execução de pipelines potencialmente longas, garantindo consistência do estado, robustez perante mensagens assíncronas tardias e restauração automática do último resultado válido.

O diagrama de sequência foi concebido para refletir fielmente a implementação atual, evidenciando a interação entre frontend, API Gateway, serviço de projetos, sistema de mensageria e workers de processamento. Em particular, ilustra-se o mecanismo de controlo por tokens (`textit{activeToken}`, `textit{committedToken}` e `textit{prevCommittedToken}`), que permite invalidar execuções em curso, ignorar outputs obsoletos e evitar que resultados de processamentos cancelados sejam apresentados ao utilizador.

Para além do fluxo principal de cancelamento, a vista inclui ainda os principais casos condicionais associados à concorrência (conflitos de versão), à devolução de operações avançadas e à receção assíncrona de resultados provenientes dos workers. Desta forma, o diagrama sintetiza o comportamento end-to-end do sistema no cenário de cancelamento, desde a ação do utilizador até à estabilização final do estado apresentado na interface.

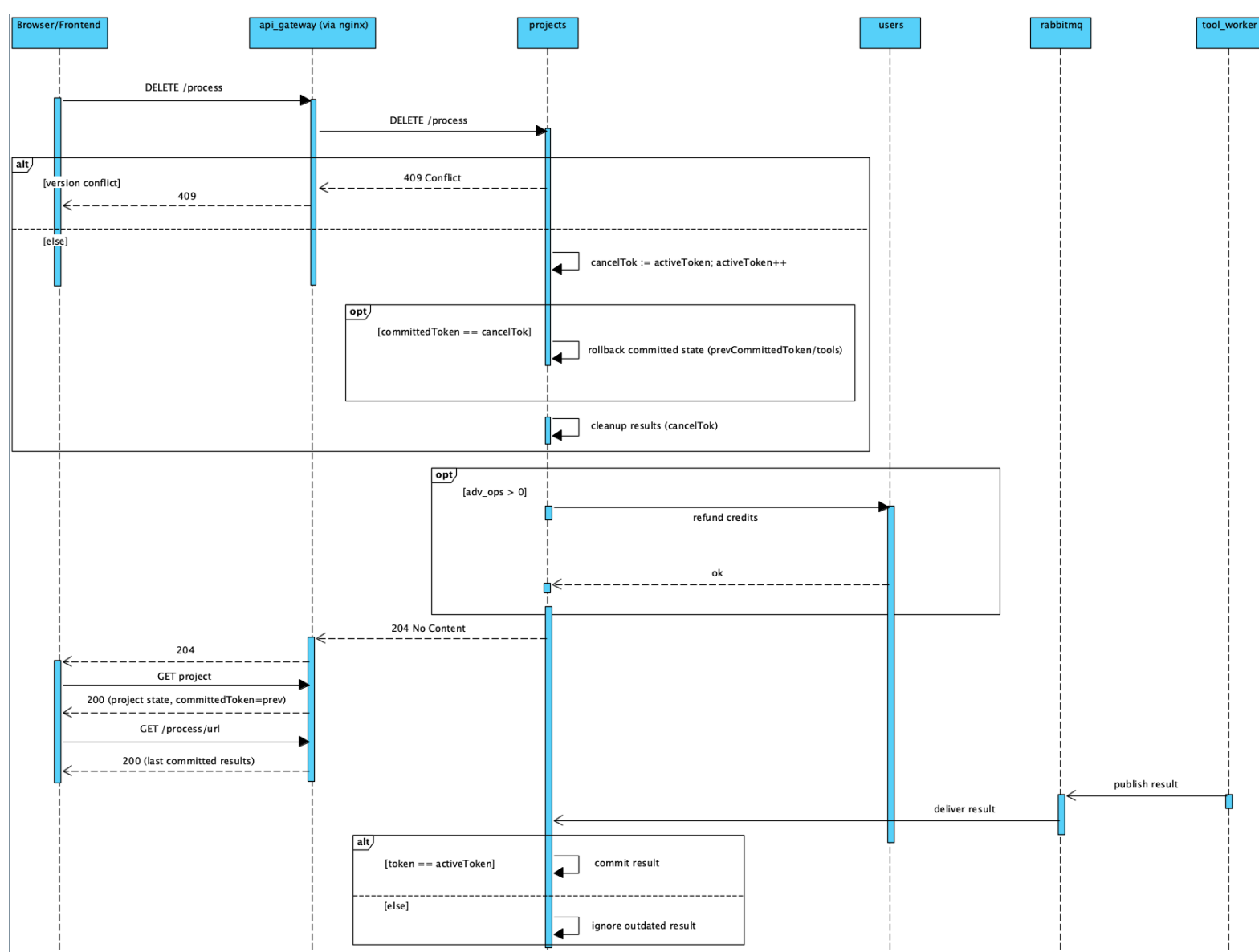


Figura 3: Diagrama de Sequência - Cancelamento

O fluxo inicia-se com o frontend a emitir um pedido `textit{DELETE /process}`, que é encaminhado pelo `textit{api_gateway}` para o serviço `textit{projects}`. Caso seja detetado um conflito de versão (controlo de concorrência otimista), o pedido é rejeitado com `textit{HTTP 409 Conflict}`, evitando cancelamentos sobre um estado desatualizado.

Na ausência de conflitos, o serviço `textit{projects}` captura o token da execução ativa (`textit{cancelTok = activeToken}`) e incrementa imediatamente o `textit{activeToken}`. Esta operação invalida todos os outputs assíncronos que ainda possam estar em trânsito, garantindo que mensagens tardias provenientes dos workers nunca serão consideradas válidas após o cancelamento.

Se o token cancelado corresponder ao último processamento confirmado (`textit{committedToken == cancelTok}`), é efetuado um rollback explícito do estado comprometido, restaurando `textit{committedToken}` e os resultados associados para `textit{prevCommittedToken}`. Este mecanismo permite que o sistema reapresente automaticamente o último conjunto de resultados válido, sem necessidade de novo processamento.

De seguida, o backend remove exclusivamente os resultados associados ao token cancelado, preservando execuções anteriores já concluídas. Em paralelo, caso existam operações avançadas associadas à execução em curso, é efetuado um reembolso seletivo através do serviço `textit{users}`, garantindo que apenas as operações efetivamente reservadas para essa execução são devolvidas.

O pedido de cancelamento termina com sucesso (`textit{HTTP 204 No Content}`), permitindo ao frontend retomar imediatamente o estado normal da interface. Para garantir coerência visual, o frontend recarrega o estado do projeto e os resultados associados ao `textit{committedToken}` atual, voltando a apresentar o último processamento válido.

Por fim, o diagrama evidencia o comportamento assíncrono do sistema: resultados publicados tardiamente pelos `textit{tool workers}` através do `textit{rabbitmq}` são entregues ao serviço `textit{projects}`, que valida cada mensagem contra o `textit{activeToken}`. Outputs cujo token não corresponda ao token ativo são explicitamente ignorados, enquanto apenas resultados com token válido podem ser comprometidos.

Este desenho assegura que o cancelamento é robusto, não destrutivo e consistente, evitando estados intermédios inválidos, reembolsos incorretos ou apresentação de resultados obsoletos. A lógica de tokens centraliza a responsabilidade da coerência no backend, tornando o sistema tolerante a atrasos, reordenação de mensagens e falhas parciais típicas de arquiteturas distribuídas.

11. Deployment View

Esta secção apresenta a **Deployment View** do sistema, descrevendo o mapeamento dos principais componentes de software na infraestrutura de execução local. A figura sintetiza, num único cenário, os nós relevantes ao funcionamento do sistema em ambiente de desenvolvimento: ponto de entrada (nginx), interface web (frontend), gateways (api_gateway e ws_gateway), serviços de domínio (projects, users, subscriptions), infraestrutura de mensageria (rabbitmq), armazenamento de objetos (minio), persistencia (MongoDB por serviço) e serviços auxiliares (img_storage, backups e mock generativo).

O objetivo deste diagrama é tornar explícito **onde** cada componente executa e **que tipos de ligações** existem entre eles (HTTP/REST, WebSocket/Socket.IO, AMQP, S3 API e MongoDB), mantendo a direcionalidade e detalhes de portas como informação complementar descrita no texto após a figura.

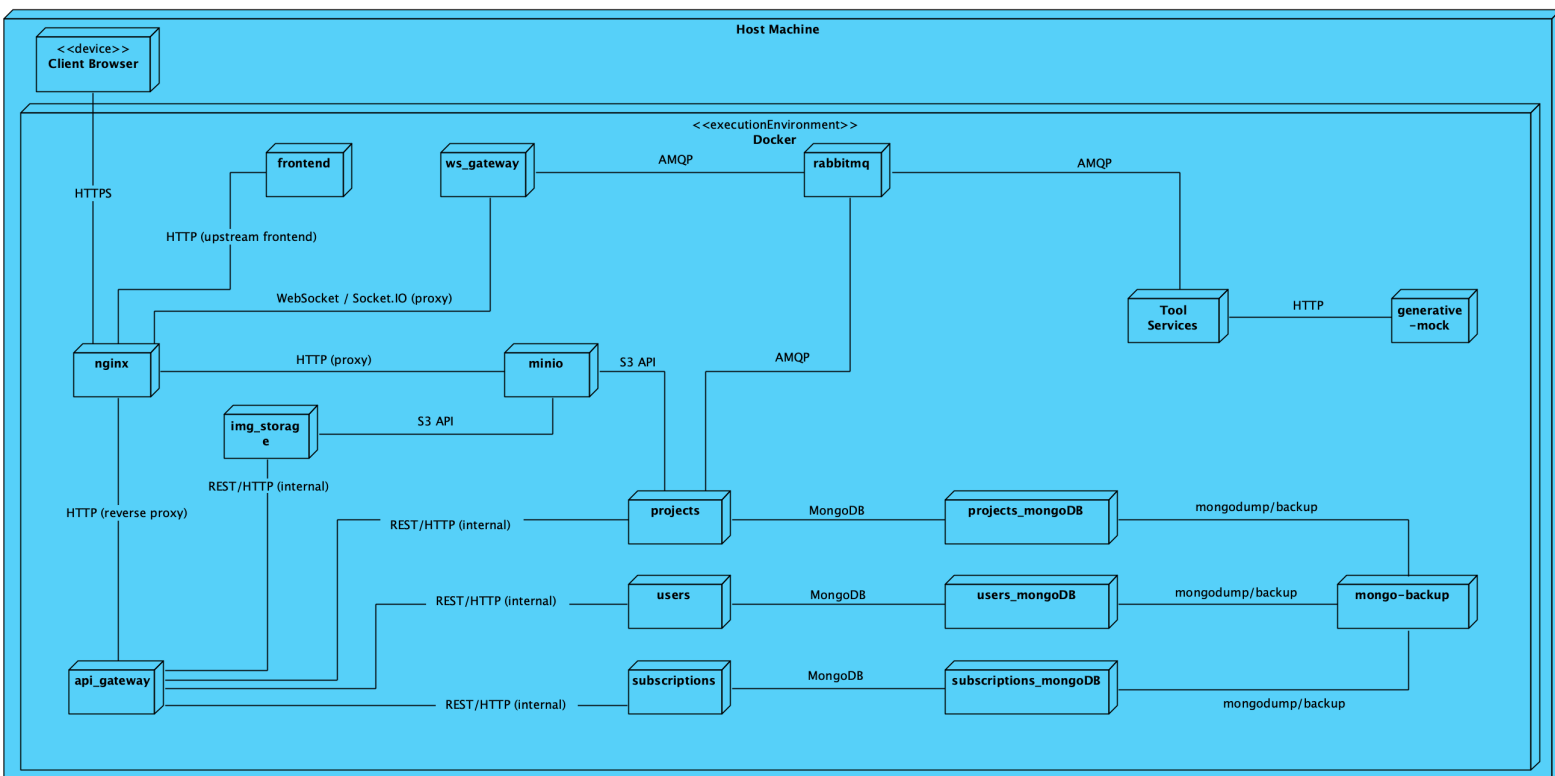


Figura 4: Diagrama de Instalação - Deployment View

Na figura, o **Host Machine** representa a máquina do utilizador (ambiente local). Dentro do **executionEnvironment Docker** encontram-se os containers que executam os componentes do sistema. O **Client Browser** encontra-se fora do ambiente Docker e comunica com o sistema através do **nginx**.

Ponto de entrada e UI. O **nginx** funciona como ponto unico de entrada, terminando TLS para o cliente (**HTTPS**) e encaminhando: (i) tráfego **HTTP** para o **frontend** (servido como upstream), (ii) tráfego **HTTP** para o **api_gateway** (reverse proxy para as rotas da API), e (iii) tráfego **WebSocket/Socket.IO** (proxy) para o **ws_gateway**. Apesar de a figura não apresentar direções, assume-se que todas estas ligações partem do browser para o **nginx**, e do **nginx** para os respetivos upstreams internos.

API Gateway e chamadas sincrona (REST). O **api_gateway** encaminha chamadas REST/HTTP internas para os serviços de domínio: **projects**, **users** e **subscriptions**. Estas ligações representam chamadas sincrona de negocio (ex.: carregamento/atualização de projeto, autenticação e operações de conta/subscrição). A figura não explicita headers/contexto (ex.: autenticação, versão do projeto, contexto de partilha), pelo que estes detalhes pertencem ao contrato das rotas e são tratados ao nível da aplicação.

Tempo real (Socket.IO). O **ws_gateway** representa o gateway de tempo real baseado em Socket.IO. O cliente estabelece a ligação Socket.IO através do nginx (proxy). A associação de clientes a “salas” de projeto e a emissão de eventos (ex.: atualizações de colaboração) é responsabilidade do **ws_gateway**, mas esses pormenores não ficam visíveis no diagrama por ser uma vista de deployment.

Mensageria (RabbitMQ / AMQP). O **rabbitmq** atua como broker AMQP. Nesta vista, ele suporta dois tipos de fluxos: (1) eventos de tempo real (publicados por serviços de domínio e consumidos pelo **ws_gateway**), e (2) pedidos/resultados de processamento (publicados por serviços de domínio e consumidos por workers/ferramentas). A figura mostra ligações AMQP entre rabbitmq e os produtores/consumidores (**ws_gateway** e Tool Services), mas não explicita filas/exchanges concretos (por exemplo, “ws_queue”, “project_queue” e filas por ferramenta). Esse detalhe é intencionalmente omitido aqui e pertence à vista de runtime/processamento.

Ferramentas e mock generativo. O bloco **Tool Services** agrega os serviços/workers responsáveis por executar ferramentas (incluindo as de IA). Para o caso do **Generative Expand**, existe um serviço **generative-mock** que expõe um endpoint HTTP usado pelo(s) worker(s) de **expand_ai** em ambiente local. O diagrama não explicita o caminho do endpoint (ex.: “/outpaint”) nem a variável de configuração usada para apontar para esse serviço; estes detalhes ficam no nível de configuração e contrato HTTP do tool.

Armazenamento de objetos (MinIO) e img_storage. O **minio** fornece armazenamento de objetos via API compatível com S3. O componente **img_storage** representa um serviço auxiliar para operações internas sobre ficheiros/imagens e comunica com o minio via S3 API. Adicionalmente, o serviço **projects** depende do minio para obter/armazenar imagens e resultados (também via S3 API), embora a figura abstraia quaisquer buckets e URLs temporárias.

Persistencia (MongoDB por serviço). Cada microserviço de domínio possui a sua própria instância MongoDB: **projects_mongoDB**, **users_mongoDB** e **subscriptions_mongoDB**. As ligações rotuladas como “MongoDB” representam ligações diretas do serviço para a base de dados correspondente, reforçando isolamento de dados por domínio.

Backups. O serviço **mongo-backup** executa backups periódicos (mongodump) das instâncias MongoDB. A figura mostra as ligações de backup para cada base de dados, mas não explicita política de retenção, agendamento exato, formato de ficheiro ou volume partilhado; estes detalhes pertencem à configuração de infraestrutura.

Informação não explicita no diagrama (assumida pelo sistema):

- todos os componentes dentro do bloco Docker executam na mesma rede de containers e comunicam por nomes de serviço DNS internos;
- o nginx e o único ponto exposto ao browser (terminação TLS e encaminhamento por paths);
- as ligações “REST/HTTP (internal)” ocorrem apenas dentro da rede Docker (não expostas diretamente ao cliente);
- o RabbitMQ é usado para comunicação assíncrona e pode reentregar mensagens; a consistência é tratada ao nível do protocolo de aplicação (ex.: idempotência/deduplicação em runtime views);

- o MinIO é acessado via S3 API com credenciais/configuração de ambiente, e o acesso externo (se existir) é feito via nginx/proxy conforme configuração.

12. Análise Crítica

ID	Descrição	Prioridade	Estado	Observações
T-01/T-10	Permitir cancelar o processamento sem recarregar.	Alta	Implementado	Implementado com modelo de execução por tokens (activeToken, committedToken e prevCommittedToken), permitindo rollback automático para o último processamento válido e remoção seletiva de resultados ao cancelar. Garante que resultados de execuções canceladas nunca são apresentados.
T-02	Uniformizar mensagens de erro, tornando-as claras e informativas, indicando causa e ação de resolução. Incluir erros da IA e de rede.	Média	Implementado	Criação de um módulo centralizado que converte códigos técnicos em mensagens funcionais em português com sugestões de resolução.
T-03	Garantir tempos de aplicação dos filtros conforme requisitos e otimizar quando necessário.	Alta	Implementado	Monitorização end-to-end (frontend + backend + fila + tools) com limiares distintos (5s sem IA, 60s com IA) e, adicionalmente, otimização por short-circuit no Apply: quando o pipeline não mudou desde o último commit, o Projects-MS evita recomputação e não publica jobs no RabbitMQ, notificando imediatamente o cliente.
T-04	Corrigir diferenças entre pré-visualização e resultado final, garantindo correspondência total.	Média	Implementado	Correção da pipeline de processamento e gestão de ficheiros, garantindo correspondência total entre a pré-visualização apresentada e o resultado final produzido.
T-05	Encriptar dados em trânsito e repouso e permitir revogação imediata de links de partilha.	Alta	Implementado	Dados sensíveis cifrados na base de dados com AES-256 e integridade das comunicações garantida por TLS 1.3/Nginx

ID	Descrição	Prioridade	Estado	Observações
T-06	Permitir reordenar ferramentas e reaplicar a nova sequência com atualização imediata.	Alta	Implementado	Reordenação aplicada de forma imediata no estado do projeto e pipeline de execução, mantendo o processamento final como ação explícita para evitar consumo desnecessário de recursos.
T-07	Validar limite antes de processar, atualizar contador após cada operação e mostrar operações restantes.	Alta	Implementado	Validação do limite antes da execução, atualização consistente após processamento ou cancelamento e apresentação correta das operações restantes ao utilizador.
T-08	Ao remover conta, apagar/anonimizar dados, projetos e links; pedir confirmação; cumprir RGPD.	Alta	Implementado	Implementado um fluxo completo de remoção de conta com eliminação de dados em todos os serviços, cumprindo integralmente os requisitos do RGPD.
T-09	Permitir que o utilizador consiga escolher em que formato pretende fazer download do projeto	Alta	Implementado	Download alargado a múltiplos formatos, com correção de problemas anteriores e melhoria significativa da usabilidade.
T-11	Implementar backups automáticos diários dos dados do utilizador para permitir restaurar os mesmos em caso de falha ou perda de dados	Alta	Implementado	Implementação de backups automáticos diários das bases de dados, com validação por testes de restauração, garantindo robustez e recuperação de dados.

Tabela 5: Tabela das tarefas implementadas relativas ao UC07

ID	Descrição	Prioridade	Estado	Observações
RF41	Edição colaborativa em tempo real.	Should	Implementado	Edição concorrente baseada em propagação de operações via WebSockets, com sincronização imediata, fallback de resincronização e recuperação automática de conflitos por versionamento otimista.
RF45	Partilhar projetos por link seguro.	Should	Implementado	Implementação integral utilizando UUIDs para garantir o anonimato e a segurança dos links gerados
RF48	Acesso à Partilha	Must	Implementado	Acesso controlado no backend com distinção entre utilizador autenticado e contexto de partilha.
RF50	Forçar Guardar	Must	Implementado	Implementado para evitar perda de alterações antes de operações destrutivas ou de processamento.
RF51	Seleção de Permissão.	Must	Implementado	Permissões read/edit configuráveis no momento da criação do link de partilha.
RF52	Geração de Link.	Must	Implementado	Geração de links únicos com identificador próprio e associação direta ao projeto.
RF53	Acesso por Link.	Must	Implementado	O acesso por link respeita a permissão definida, ativando ou restringindo ferramentas conforme o modo, com comportamento consistente.
RF54	Acesso à Gestão de Links.	Should	Implementado	Acesso à gestão de links disponível apenas para o proprietário do projeto.
RF55	Revogar Acesso.	Should	Implementado	O proprietário consegue invalidar links específicos através do painel de gestão, com efeito imediato nas permissões da base de dados.
RF56	Invalidação de Link.	Should	Implementado	Após a revogação, o sistema bloqueia o acesso e devolve o estado, encerrando sessões ativas.

ID	Descrição	Prioridade	Estado	Observações
RF57	Gestão de Erro de Link.	Must	Implementado	O sistema apresenta mensagens claras sobre links inválidos ou revogados.
RNF53	Gestão de Concorrência.	Must	Implementado	O sistema detecta e resolve conflitos de versão sem perda de dados.
RNF65	Limite de utilizadores num Projeto.	Should	Implementado	O mecanismo de Presence garante o limite de 2 colaboradores em simultâneo para utilizadores não premium.

Tabela 7: Tabela dos requisitos implementados relativos ao UC07

As tabelas apresentadas demonstram que os requisitos e tarefas associados ao UC07 — Partilha e Colaboração foram implementados de forma consistente e completa, cobrindo a totalidade das funcionalidades e preocupações identificadas na fase de análise. Todas as tarefas definidas foram concretizadas, evidenciando uma implementação alinhada com os objetivos do caso de uso.

13. Conclusão

Ao longo deste trabalho foi desenvolvida e analisada a evolução e expansão do sistema PictuRAS, com destaque na correção de problemas e inconsistências identificadas na solução inicial, na introdução de novas funcionalidades e na definição de uma arquitetura capaz de suportar essas evoluções de forma consistente. As tarefas de manutenção corretiva permitiram resolver limitações relevantes ao nível da experiência do utilizador e da coerência do sistema, nomeadamente no cancelamento de processamentos, na uniformização das mensagens de erro e na gestão de estados intermédios.

A implementação de novos casos de uso, em particular os relacionados com partilha e colaboração, demonstrou a importância de mecanismos de comunicação assíncrona e de sincronização eficiente entre frontend e backend. A utilização de WebSockets, aliada a estratégias de controlo de concorrência e versionamento otimista, permitiu garantir consistência dos dados sem comprometer a fluidez da interação entre utilizadores.

Do ponto de vista arquitetural, a adoção de uma abordagem baseada em micro-serviços revelou-se adequada para responder aos requisitos funcionais e não funcionais identificados. A separação clara de responsabilidades, a persistência independente por serviço e a integração de componentes de suporte, como sistemas de mensagens e armazenamento de objetos, contribuíram para uma solução mais modular, escalável e robusta, facilitando a evolução do sistema e a introdução de novas funcionalidades, como o Assistente de IA.

Como trabalho futuro, identificam-se várias oportunidades de melhoria. Em particular, o módulo de funcionalidades generativas poderá beneficiar da integração com modelos mais avançados e da utilização de análise automática de conteúdo da imagem, permitindo gerar sugestões mais contextuais e menos baseadas em heurísticas. Da mesma forma, o suporte à colaboração em tempo real poderá ser aprofundado com mecanismos de edição verdadeiramente concorrente e sincronização de estado entre múltiplos utilizadores, ultrapassando o modelo atual centrado em sessões independentes. Estas extensões permitiriam tornar o sistema mais inteligente, colaborativo e próximo de uma plataforma profissional de edição distribuída.

Em suma, o trabalho realizado evidencia a importância de alinhar decisões arquiteturais com os requisitos do sistema, demonstrando como uma arquitetura bem estruturada pode melhorar a manutenção, a escalabilidade e a qualidade global da solução, ao mesmo tempo que cria bases sólidas para a sua evolução futura.